Redes de Comunicação — Resumo Teórico

Gonçalo Sousa

Atualizado: Semana 3 (Capítulos 1–2 + Ficha 3)

Conteúdo

Informação da Disciplina **3**

Aula 1 — Introdução às Redes de Comunicação **4**
O que é a Internet? 4
Extremidade da rede (edge) 5
Núcleo da rede (core) 6
Atrasos, perdas e *throughput* 11
Camadas protocolares e modelos de serviço 17

Aula 2 — Camada de aplicação **21**
Princípios das aplicações em rede 21
Web e HTTP 24
FTP 32
Correio eletrónico 33
DNS 38
Aplicações P2P 44
Programação com sockets 46
Resumo comparativo dos protocolos 52

Informação da Disciplina

Notas de organização da cadeira (não é matéria de exame, mas é útil ter aqui para consulta rápida em vez de andar a abrir o PDF `Teoricas/Aula_01.pdf`).

Docentes: Rui Prior (rcprior@fc.up.pt, gabinete 1.33) e Pedro d'Orey (pdorey@fc.up.pt).

Avaliação: avaliação contínua — 3 testes ao longo do semestre (cada um com componente teórica e prática, cada um vale 30% da nota final) + um trabalho de programação em grupos de 2 (10% da nota final). O exame de recurso substitui os testes.

Requisitos: disciplina introdutória e praticamente auto-contida. Precisa-se de conhecimentos básicos de Java para a parte de programação com sockets (mais para a frente no semestre).

Bibliografia principal: *Computer Networking: A Top-Down Approach*, Jim Kurose e Keith Ross (Addison-Wesley) — os slides das teóricas são traduzidos e adaptados deste livro. Alternativas online gratuitas: *Computer Networking: Principles, Protocols and Practice* (Olivier Bonaventure) e *Fundamentos de Redes de Computadores* (José Legatheaux Martins).

Programa da disciplina (para situar onde estamos): 1. Introdução (arquiteturas de rede, comutação, camadas ISO/OSI e TCP/IP) — **é o que cobre esta Aula 1** — 2. Camada de Aplicação (HTTP, FTP, SMTP, DNS, sockets) — 3. Camada de Transporte (UDP, TCP, controlo de fluxo/ congestionamento) — 4. Camada de Rede (encaminhamento, IP, ICMP, DHCP, NAT, RIP/OSPF/BGP) — 5. Camada de Ligação de Dados (deteção/correção de erros, Ethernet, ARP, redes sem fios).

Aula 1 — Introdução às Redes de Comunicação

O que é a Internet?

Visão estrutural: hosts, ligações e routers

A Internet pode ser descrita de duas formas complementares: pelos seus **componentes** (visão estrutural, "debaixo do capô") ou pelo **serviço** que presta às aplicações.

Do ponto de vista estrutural, a Internet é composta por:

- **Terminais** (*hosts* ou *end systems*): milhões de dispositivos ligados à rede — não só PCs e servidores, mas cada vez mais objetos do dia-a-dia ("IoT" — frigoríficos, torradeiras, carros, câmaras de vigilância, telefones IP). Todos correm **aplicações em rede**.
- **Ligações** (*links*): o meio por onde os dados viajam — fibra, cobre, rádio, satélite. Cada ligação tem uma **taxa de transmissão** (também chamada capacidade ou, informalmente, "largura de banda"), medida em bits por segundo.
- ***Routers***: dispositivos que reenviam pacotes de dados, encaminhando-os em direção ao seu destino.

Definição — Internet

A Internet é uma **"rede de redes"**: uma coleção de redes independentes interligadas, vagamente organizada de forma hierárquica (ver secção sobre a estrutura da Internet, mais à frente). Distingue-se a **Internet pública** (a que todos conhecemos) de **intranets privadas**, que usam a mesma tecnologia mas não estão acessíveis a partir da Internet pública (ou só o estão de forma restrita).
Toda a comunicação na Internet é regida por **protocolos** — por exemplo, TCP e IP controlam o envio e receção de pacotes; HTTP rege a Web; Ethernet rege a transferência de dados numa rede local. Estes protocolos são definidos em documentos chamados **RFC** (*Request for Comments*), geridos pelo **IETF** (*Internet Engineering Task Force*).

Visão de serviços

Vista do lado das aplicações, a Internet é uma **infraestrutura de comunicação** que possibilita a existência de aplicações distribuídas: Web, email, VoIP, jogos em rede, partilha de ficheiros, etc. Estas aplicações não precisam de saber como os bits chegam ao destino — só precisam de um serviço de comunicação. A Internet oferece dois tipos:

- **Entrega de dados fiável**, extremo-a-extremo (usada por TCP);
- **Entrega de dados não-fiável**, do tipo "melhor esforço" (*best effort*, usada por UDP).

Estes dois modelos de serviço estão explicados em detalhe na Aula 2 ("Serviços de transporte da Internet: TCP e UDP").

O que é um protocolo?

Definição — Protocolo

Um **protocolo** define:

- o **formato** das mensagens trocadas;
- a **ordem** de envio e receção dessas mensagens entre as entidades da rede;
- as **ações** a efetuar na transmissão ou na receção de uma mensagem (ou em resposta a outro evento, como um temporizador que expira).

É exatamente como um protocolo humano: quando dizemos "Olá" a alguém e recebemos "Olá" de volta antes de perguntar as horas, estamos a seguir um protocolo (um formato de mensagem — uma saudação — e uma ordem — a saudação antes do pedido).

Exemplo resolvido

Paralelo entre um protocolo humano e um protocolo de rede.
Do lado humano: a pessoa A diz "Olá" à pessoa B; B responde "Olá"; só depois é que A pergunta "Tens horas?" e B responde "São 2:00". Se B, em vez de "Olá", respondesse logo com a hora, a comunicação seria estranha — quebrava-se o protocolo esperado (a ordem das mensagens importa).
Do lado da rede, o mesmo tipo de troca acontece quando um browser vai buscar uma página web:

1. O cliente (o browser) envia um **pedido de ligação TCP** ao servidor.
2. O servidor responde com uma **resposta de ligação TCP**, confirmando que aceita estabelecer a conexão (isto é o "handshake" — o estabelecimento prévio da ligação; ver Aula 2).
3. Só depois de a ligação estar estabelecida é que o cliente envia o pedido real, por exemplo `GET http://www.exemplo.com/pagina.html`.
4. O servidor responde enviando o `<ficheiro>` pedido.

Tal como no protocolo humano, a **ordem importa**: o cliente não pode enviar o `GET` antes de a ligação TCP estar estabelecida, tal como B não responde "são 2 horas" antes de ouvir a pergunta. E o **formato** também importa: `GET http://...` é uma mensagem com uma sintaxe específica que o servidor sabe interpretar — se o cliente enviasse uma mensagem com outro formato, o servidor não saberia o que fazer com ela.

Extremidade da rede (edge)

Terminais, aplicações e serviços de transporte

Os terminais (*hosts*) correm as aplicações (Web, email, etc.) e ficam na periferia ("extremidade") da rede. Os slides desta aula apresentam aqui os dois modelos de organização das aplicações (**cliente/servidor** e ***peer-to-peer***) e os dois serviços de transporte que a Internet oferece às aplicações (**TCP** e **UDP**). O Capítulo 2 volta a estes temas com muito mais detalhe, por isso estão explicados de uma só vez na Aula 2: secções "Arquiteturas das aplicações" e "Serviços de transporte da Internet: TCP e UDP".

Redes de acesso residenciais

A **rede de acesso** é o que liga um terminal ao primeiro *router* da rede (o *edge router*). Os aspetos relevantes de qualquer rede de acesso são a sua **capacidade** (partilhada ou dedicada) e a sua **latência**.

Definição

Acesso por cabo (HFC — *hybrid fiber/coaxial*): a mesma infraestrutura de cabo coaxial da TV por cabo transporta também dados. Usa-se **multiplexagem em frequência** (FDM): canais de TV e canais de dados (subida e descida) são transmitidos em bandas de frequência diferentes no mesmo cabo — daí ser possível ver TV e ter Internet ao mesmo tempo pelo mesmo fio. É uma ligação **assimétrica** (mais capacidade a descarregar do que a enviar, tipicamente até ~30Mbps a descer e ~2Mbps a subir) e o cabo é **partilhado** entre todas as casas ligadas ao mesmo segmento — ou seja, o débito real de cada casa depende de quantos vizinhos estão a usar a rede ao mesmo tempo.
Fibra até casa (FTTH — *fiber to the home*): ligação ótica direta (ou quase) da central até casa, através de um *splitter* ótico que distribui o sinal de uma fibra para várias casas (rede **PON**, passiva) ou com eletrónica ativa por casa (**AON**). Capacidade muito superior à do cabo e a distâncias maiores, e permite integrar telefone e televisão na mesma fibra.

Atenção

Nota adicional (não estava explícito nos slides): a diferença central entre cabo (HFC) e fibra (FTTH) não é só "a fibra é mais rápida" — é que no **cabo o meio físico é partilhado** entre vizinhos (o mesmo segmento de cabo coaxial serve várias casas, multiplexado em frequência), enquanto na **fibra (PON)** cada casa tem uma fibra dedicada até ao *splitter*, e a partilha (se existir) acontece de forma mais controlada mais a montante, na central. Isto tem implicações práticas: numa rede de cabo, se muitos vizinhos estiverem a fazer *streaming* ao mesmo tempo, o teu débito pode descer — na fibra, isso é bem menos provável.

Redes de acesso empresariais e sem fios

Definição

LANs empresariais (Ethernet): nas empresas e universidades, os terminais ligam-se ao *router* através de uma rede local Ethernet, a diferentes velocidades padronizadas (10 Mbps, 100 Mbps, 1 Gbps, 10 Gbps). Na configuração moderna, os terminais ligam-se a um **switch Ethernet** (em vez de partilharem um cabo comum, como acontecia nas Ethernet mais antigas).
Redes sem fios (*wireless*): uma rede de acesso sem fios partilhada liga os terminais a um *router* através de um **ponto de acesso**. Nas LANs sem fios (WiFi, normas 802.11b/g/n/ac/ax/be), o débito máximo teórico varia entre 11 Mbps (802.11b, a norma mais antiga) e 46 Gbps (802.11be, a mais recente). Nas redes de longa distância sem fios (WAN sem fios), operadas por operadoras de telecomunicações, os débitos vão até ~300 Mbps em 4G (LTE Avançado) e até 20 Gbps em 5G.

Definição

Meio físico — guiado vs. não guiado. Um bit propaga-se entre um par emissor/recetor através de uma **ligação física**. Há dois tipos:

- **Meio guiado:** o sinal propaga-se por um meio sólido — par de cobre entrançado (categorias Cat.5/5e/6/6a, suportando de 100 Mbps a 10 Gbps de Ethernet consoante a categoria), cabo coaxial, ou fibra ótica.
- **Meio não guiado:** o sinal propaga-se livremente pelo ar/espaço — ondas de rádio. Tipos de ligação rádio: microondas terrestres (até 620 Mbps), LAN sem fios (WiFi, até 46 Gbps), redes de longa distância (redes móveis, até 20 Gbps em 5G) e satélite (geossíncrono ou de baixa altitude — os satélites de baixa altitude, como a constelação Starlink, têm latência muito menor: cerca de 25ms contra 270ms dos geossíncronos, mas capacidade menor por satélite, até ~220 Mbps).

O sinal transportado num meio pode ser transmitido em **banda-base** (um único canal ocupa toda a capacidade do meio, sem modulação — é o caso da Ethernet original em cabo coaxial) ou **modulado** (o sinal digital modula uma onda portadora em frequência, amplitude ou fase — permitindo usar múltiplos canais no mesmo meio, como no HFC).

Núcleo da rede (core)

O **núcleo da rede** é a malha de *routers* interligados que liga as diferentes redes de acesso entre si. A questão fundamental do núcleo da rede é: **como se efetua a transferência de dados através da rede?** Há duas respostas possíveis — comutação de circuitos e comutação de pacotes.

Comutação de circuitos

Definição — Comutação de circuitos

Na comutação de circuitos (como na rede telefónica tradicional), os recursos necessários ao longo de todo o percurso (capacidade nas ligações, capacidade de comutação nos nós) são **reservados extremo-a-extremo antes** de a comunicação começar — é necessário **estabelecer previamente** a "chamada". Uma vez reservados, esses recursos ficam garantidos e **não são partilhados** com mais ninguém durante a duração da chamada, mesmo que não haja nada para transmitir nesse momento (desperdício de recursos). Em compensação, o **desempenho fica garantido** — não há congestionamento possível dentro daquele circuito.

Os recursos são divididos em "parcelas" atribuídas às chamadas, através de duas técnicas de multiplexagem:

- **FDM** (*Frequency Division Multiplexing*): a capacidade total é dividida em bandas de frequência, uma por utilizador — cada utilizador usa a "sua" banda em permanência, durante toda a chamada.
- **TDM** (*Time Division Multiplexing*): o tempo é dividido em ciclos, e cada ciclo é dividido em *slots* — cada utilizador usa o "seu" *slot* em cada ciclo, sempre o mesmo, de forma periódica e determinística.

Exemplo resolvido

Cálculo do tempo de transferência num circuito. Quanto tempo demora a enviar um ficheiro de 640 000 bits do terminal A para o terminal B, numa rede de comutação de circuitos, sabendo que:

- as ligações ao longo do percurso têm capacidade de 2,048 Mbps;
- cada ligação usa TDM com 32 *slots* por ciclo (ou seja, a capacidade total da ligação é dividida em 32 fatias iguais, uma por utilizador possível);
- são necessários 500 ms para estabelecer o circuito extremo-a-extremo (o *handshaking* prévio, antes de poder começar a enviar dados).

Passo 1 — capacidade efetivamente atribuída ao circuito. Como a capacidade da ligação (2,048 Mbps) é dividida em 32 *slots* iguais, e o nosso circuito só usa 1 desses *slots*, a capacidade que o nosso circuito efetivamente tem disponível é:

$$\frac{2{,}048 \text{ Mb/s}}{32} = 64 \text{ kb/s}$$

Passo 2 — tempo de transmissão dos dados, usando essa capacidade de 64 kb/s:

$$\frac{640\,000 \text{ bits}}{64\,000 \text{ bits/s}} = 10 \text{ s}$$

Passo 3 — somar o tempo de estabelecimento do circuito (os 500 ms gastos antes de o circuito estar pronto, que têm de ser somados ao tempo de transmissão dos dados, porque acontecem em sequência, não em simultâneo):

$$10 \text{ s} + 500 \text{ ms} = \mathbf{10{,}5} \text{ s}$$

(Na prática, seria ainda preciso somar o tempo de propagação do sinal ao longo do circuito — ver a secção "Atrasos, perdas e *throughput*" mais à frente, onde este conceito é aprofundado.)

Comutação de pacotes

Definição — Comutação de pacotes

Na comutação de pacotes, o fluxo de dados extremo-a-extremo é dividido em pequenos pedaços chamados **pacotes** (dados da aplicação + informação de controlo/cabeçalho). Ao contrário da comutação de circuitos:

- **Não há reserva prévia de recursos.** Pacotes de diferentes utilizadores partilham os recursos da rede à medida da necessidade — é a chamada **multiplexagem estatística**: a sequência de pacotes de cada utilizador não segue nenhum padrão fixo (ao contrário do TDM, em que cada emissor usa sempre o mesmo *slot*, de forma determinística e periódica); a capacidade é distribuída consoante a procura em cada instante.
- Cada transmissão de um pacote usa a **capacidade total** da ligação (não uma fração fixa, como no TDM).
- Como não há reserva, pode haver **contenda** no acesso: se a procura de recursos por parte de vários utilizadores exceder a oferta disponível num dado instante, os pacotes ficam em **fila de espera** e pode ocorrer **congestionamento**.

Definição — Store-and-forward

Os pacotes são transferidos **salto a salto** (de *router* em *router*) ao longo do percurso. Em cada nó (*router*), aplica-se a regra *store-and-forward*: o *router* tem de **receber o pacote completo** antes de poder começar a reenviá-lo para o salto seguinte — não pode começar a retransmitir um pacote que ainda não recebeu por inteiro. Isto introduz um **atraso de transmissão** em cada salto: um pacote de L bits demora L/R segundos a ser "colocado" numa ligação de capacidade R bits/s.
Como cada pacote é encaminhado de forma **independente**: cada nó escolhe o próximo salto para cada pacote individualmente, pelo que pacotes com o mesmo destino não têm de seguir a mesma rota (embora normalmente sigam), podem chegar fora de ordem, e podem mesmo perder-se ou ser danificados em trânsito — cabe às máquinas de origem e destino detetar e corrigir estas situações (é isso que o TCP faz, por exemplo).

Exemplo resolvido

Atraso *store-and-forward* ao longo de vários saltos. Um pacote de $L = 10$ Mbits é enviado de um terminal, através de dois *routers* intermédios (logo, 3 ligações/saltos no total), até ao terminal de destino. Todas as ligações têm capacidade $R = 2$ Mb/s. Qual o atraso total até o pacote chegar completo ao destino (ignorando atraso de propagação)?
Passo 1 — atraso de transmissão por salto. Em cada ligação, o pacote demora $L/R = 10\,000\,000/2\,000\,000 = 5$ segundos a ser totalmente colocado na ligação.
Passo 2 — porque é que os atrasos se somam (não se sobrepõem). Por causa da regra *store-and-forward*, o primeiro *router* só pode começar a reenviar o pacote para o segundo *router* depois de o ter recebido **na íntegra** — ou seja, depois dos 5 segundos que o terminal de origem demorou a transmiti-lo todo. Só nesse momento é que o primeiro *router* começa a transmiti-lo na segunda ligação, o que demora outros 5 segundos. E só depois disso é que o segundo *router* pode começar a enviá-lo para o destino, mais 5 segundos. Os três intervalos de 5 segundos acontecem **um a seguir ao outro**, nunca em simultâneo (cada um só pode começar quando o anterior termina).
Passo 3 — atraso total:

$$3 \times \frac{L}{R} = 3 \times 5\text{s} = \mathbf{15}\ \text{s}$$

De forma geral, para N saltos: atraso total $= N \cdot L/R$ (assumindo atraso de propagação nulo — na prática seria preciso somar também o atraso de propagação de cada ligação, tratado na próxima secção).

Comutação de pacotes vs. comutação de circuitos

A comutação de pacotes é a que permite **mais utilizadores** partilhar a mesma rede, precisamente porque não reserva recursos que depois ficam desperdiçados quando um utilizador está inativo.

Exemplo resolvido

Exemplo comparativo. Considere uma ligação de 1 Mb/s. Cada utilizador está "ativo" (a transmitir a 100 kb/s) apenas 10% do tempo, e inativo (sem transmitir nada) nos restantes 90%.
Com comutação de circuitos: cada utilizador precisa de um circuito dedicado de 100 kb/s reservado o tempo todo (mesmo quando inativo). Com uma ligação de 1 Mb/s, cabem exatamente $1000/100 = 10$ utilizadores — nem mais um, porque não há partilha possível.
Com comutação de pacotes: como os recursos só são usados quando há dados para enviar, é possível admitir **muito mais** de 10 utilizadores, desde que seja improvável que mais de 10 estejam ativos **ao mesmo tempo**. Com 35 utilizadores, a probabilidade de haver mais de 10 ativos em simultâneo é de apenas aproximadamente 0,0004 (ou seja, praticamente nunca acontece) — logo, a rede consegue servir bem 35 utilizadores com a mesma capacidade que só serviria 10 em comutação de circuitos.

Atenção

Nota adicional (não estava explícito nos slides): de onde vem o valor 0,0004? Cada utilizador está ativo com probabilidade $p = 0{,}1$, independentemente dos outros. O número de utilizadores ativos em simultâneo, entre $N = 35$, segue uma **distribuição binomial** $X \sim \text{Bin}(N, p)$. Queremos a probabilidade de mais de 10 estarem ativos ao mesmo tempo:

$$P(X > 10) = 1 - \sum_{k=0}^{10} \binom{35}{k} p^k (1-p)^{35-k}$$

Cada termo do somatório dá a probabilidade de exatamente k utilizadores estarem ativos: $\binom{35}{k}$ conta de quantas formas se pode escolher quais k, dos 35, estão ativos; p^k é a probabilidade de esses k estarem mesmo ativos; $(1-p)^{35-k}$ é a probabilidade de todos os restantes estarem inativos. Somando essa probabilidade para $k = 0$ até $k = 10$ obtém-se a probabilidade de **no máximo** 10 estarem ativos; 1 menos esse valor dá a probabilidade de **mais de** 10 estarem ativos. Na prática, este cálculo faz-se com uma calculadora ou software (não se espera que se some os 11 termos à mão num teste) — o que importa perceber é o modelo (binomial) e a ideia de que, com muitos utilizadores independentes, é estatisticamente muito improvável que "todos" estejam ativos ao mesmo tempo.

Frequente em exame

A comutação de pacotes não é uma "panaceia" — é ótima para tráfego em rajadas (*bursty*, como a navegação Web) porque permite partilha de recursos sem estabelecimento prévio de chamada, mas pode sofrer **congestionamento**: atrasos e perdas de pacotes quando a procura excede a capacidade disponível num dado momento. Por isso são necessários **protocolos** (como o TCP) para garantir transferência de dados fiável e para controlar o congestionamento. Garantir à comutação de pacotes um desempenho tão previsível quanto o de um circuito dedicado (para fluxos áudio/vídeo, por exemplo) continua a ser, em geral, um problema em aberto. Isto é um ponto frequentemente sublinhado como importante — a ideia de que comutação de pacotes tem *trade-offs*, não é estritamente "melhor" que comutação de circuitos.

Pratica agora — Multiplexagem estatística (Ficha 2, Ex. 2)

O Ex. 2 da Ficha 2 é o exemplo acima (1 Mbps, utilizadores de 100 kbps ativos 10% do tempo). As alíneas (a) e (b) saem diretamente do exemplo. Faz as outras duas:

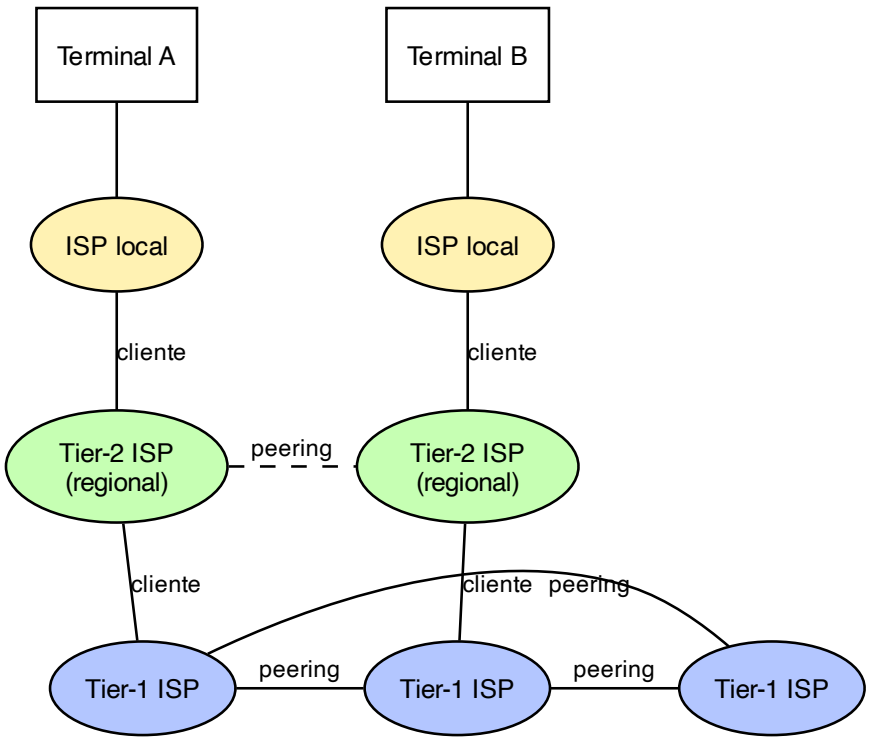
- **F2.2(c)** Com 35 utilizadores, escreve a probabilidade de exatamente k estarem a transmitir, e calcula-a para $k = 3$.
- **F2.2(d)** Calcula $P(X \geq 11)$ com a função `BINOM.DIST()` do Excel (ou uma calculadora online) e confirma o valor 0,0004 da nota acima. Cuidado: "11 ou mais" é o complementar de "10 ou menos".

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Estrutura da Internet: hierarquia de ISPs

A Internet é organizada de forma **aproximadamente hierárquica**, uma "rede de redes" de fornecedores de acesso (ISPs — *Internet Service Providers*):

- **ISPs Tier-1:** cobertura internacional, interligados entre si "todos para todos" (uma espécie de clique), através de ligações privadas chamadas **peering** — não pagam uns aos outros, trocam tráfego mutuamente porque ambos beneficiam.
- **ISPs Tier-2:** ISPs de menor dimensão, nacionais ou regionais. Ligam-se a um ou mais ISPs Tier-1 (e possivelmente a outros ISPs Tier-2). Um ISP Tier-2 tipicamente **paga** a um Tier-1 para ter conectividade ao resto da Internet — o Tier-2 é **cliente** do Tier-1 nessa relação. Alguns pares de ISPs Tier-2 também estabelecem ligações de *peering* diretamente entre si.
- **ISPs Tier-3 e locais:** as redes de acesso mais próximas dos utilizadores finais — são clientes dos ISPs de nível mais elevado, ligando-se através deles ao resto da Internet.



Um pacote enviado de um Terminal A para um Terminal B atravessa tipicamente várias destas redes: sai do ISP local de A, passa pelo ISP Tier-2 de A, eventualmente por um ou mais ISPs Tier-1 (se A e B não partilharem o mesmo Tier-2), desce pelo ISP Tier-2 de B e chega ao ISP local de B.

Atenção

Nota adicional (não estava explícito nos slides): repara na diferença entre uma ligação **cliente** e uma ligação de ***peering*** no diagrama: numa relação cliente, o ISP "de baixo" paga ao ISP "de cima" para ter conectividade ao resto da Internet (é uma relação assimétrica, como pagar a um fornecedor); numa relação de *peering*, dois ISPs do mesmo nível trocam tráfego diretamente entre si porque ambos beneficiam (evita ter de passar pelo Tier-1, que seria mais lento/caro) — é uma relação simétrica, sem pagamento entre as partes. Isto explica, por exemplo, porque é que às vezes um pacote entre dois utilizadores fisicamente próximos, mas de ISPs diferentes sem *peering* direto, pode dar uma "volta" enorme até subir e descer a hierarquia.

Pratica agora — Encaminhamento (Exercícios Semana 1, Ex. 1)

- **Ex. 1** (o posto de correios e as rotas para todos os endereços) — usa a ideia de hierarquia de ISPs desta secção: ninguém conhece o caminho completo para todos os destinos, só o próximo nível da hierarquia. Pensa também no **custo** dessa técnica (as rotas deixam de ser as mais curtas possíveis). O encaminhamento hierárquico propriamente dito só aparece mais à frente, na camada de rede.

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Atrasos, perdas e *throughput*

Nesta secção estuda-se **como e porque** os pacotes sofrem atrasos e podem perder-se numa rede de comutação de pacotes, e o que significa "débito" (*throughput*).

Os pacotes são colocados em **filas de espera** nos *routers*: se a taxa de chegada de pacotes a uma ligação de saída excede momentaneamente a capacidade dessa ligação, os pacotes ficam à espera da sua vez. Se não houver espaço livre na fila quando chega um novo pacote, este é **descartado** (perda).

As quatro fontes de atraso

Definição — As quatro componentes do atraso

O atraso total que um pacote sofre entre dois nós consecutivos ($d_{n,n+1}$) é a soma de quatro componentes:

$$d_{n,n+1} = d_{proc} + d_{fila} + d_{trans} + d_{prop}$$

1. d_{proc} — **atraso de processamento:** tempo que o nó gasta a verificar erros no cabeçalho do pacote e a determinar para que ligação de saída o deve encaminhar. Tipicamente alguns microssegundos ou menos.
2. d_{fila} — **atraso na fila de espera:** tempo que o pacote espera na fila antes de poder começar a ser transmitido na ligação de saída — é a soma dos tempos de transmissão de todos os pacotes que estão à frente na fila. Depende do nível de congestionamento do *router* naquele instante (pode variar muito, de quase zero a muito elevado).
3. d_{trans} — **atraso de transmissão:** $= L/R$, onde L é o comprimento do pacote em bits e R a capacidade da ligação em bits/s. É o tempo que demora a "colocar" todos os bits do pacote na ligação. Significativo em ligações de baixa capacidade.
4. d_{prop} — **atraso de propagação:** $= l/v$, onde l é o comprimento físico do meio (distância) e v a velocidade de propagação do sinal nesse meio (tipicamente perto de 2×10^8 m/s, cerca de 2/3 da velocidade da luz no vácuo). Pode ir de alguns microssegundos a centenas de milissegundos (por exemplo, num satélite geossíncrono).

Atenção

Nota adicional (não estava explícito nos slides): atraso de transmissão vs. atraso de propagação — não confundir! São conceitualmente muito diferentes, apesar de ambos dependerem da ligação:

- O **atraso de transmissão** (L/R) é o tempo que o **emissor** demora a empurrar todos os bits do pacote para dentro da ligação — depende do tamanho do pacote e da capacidade da ligação, **não** da distância. Pensa nisto como o tempo que um camião demora a **entrar** todo numa autoestrada através do acesso (depende do comprimento do camião e da "largura" do acesso).
- O **atraso de propagação** (l/v) é o tempo que o **sinal** demora a viajar fisicamente do emissor até ao recetor, depois de já estar todo "na ligação" — depende apenas da distância e da velocidade de propagação no meio, **não** do tamanho do pacote nem da capacidade da ligação. É o tempo que o camião (já todo na autoestrada) demora a percorrer a distância até à saída.

Um pacote muito grande numa ligação muito lenta pode ter atraso de transmissão dominante (minutos); uma ligação via satélite geossíncrono tem atraso de propagação dominante (~270ms), mesmo para um pacote minúsculo. São independentes um do outro.

Pratica agora — As quatro fontes de atraso (Ex. 6 e 7)

- **Ex. 6** — na demonstração interativa, encontra valores em que o emissor acaba de transmitir **antes** de o primeiro bit chegar ($d_{trans} < d_{prop}$) e outros em que é ao contrário.
- **Ex. 7** (o repórter "de compreensão lenta") — qual das quatro componentes explica a pausa? Estima-a para uma ligação por satélite geoestacionário (~36 000 km de altitude, sobe e desce).

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Atraso com múltiplos pacotes e ligações em série

Quando o percurso passa por várias ligações de capacidades diferentes, e são enviados vários pacotes seguidos (não só um), o comportamento depende muito de **qual ligação é o "estrangulamento"** (a mais lenta do percurso — *bottleneck link*).

Exemplo resolvido

Configuração. Um terminal T1 envia 4 pacotes, cada um com $L = 1000$ bits, para um terminal T2, passando por um único *router* R1: primeiro numa ligação LAN (T1 a R1) e depois numa ligação WAN (R1 a T2). A velocidade de propagação em ambas é $v = 2 \times 10^8$ m/s. Vamos calcular o atraso total até o **último** (4º) pacote chegar completo a T2, em dois cenários: (a) a LAN é a ligação mais lenta; (b) a WAN é a mais lenta.

Fórmulas gerais (com $d_{proc} = d_{fila} = 0$, ou seja, sem congestionamento adicional):

- Se $R_{LAN} < R_{WAN}$ (LAN é o estrangulamento):

$$d_{total} = 4 \cdot d_{trans,LAN} + d_{prop,LAN} + d_{trans,WAN} + d_{prop,WAN}$$

- Se $R_{LAN} > R_{WAN}$ (WAN é o estrangulamento):

$$d_{total} = d_{trans,LAN} + d_{prop,LAN} + 4 \cdot d_{trans,WAN} + d_{prop,WAN}$$

Porque é que o "4×" muda de posição: o fator 4 aplica-se sempre à ligação **mais lenta**. Isto acontece porque essa ligação é o gargalo: os 4 pacotes só conseguem ser colocados nela um a seguir ao outro (a ligação está sempre ocupada com o pacote anterior), pelo que o **último** pacote só começa a ser transmitido depois dos outros 3 já terem sido totalmente transmitidos nessa ligação — daí $4 \times d_{trans}$ **da ligação lenta** determinar quando o último pacote sai dela. Na ligação rápida, como cada pacote é transmitido depressa, não há essa acumulação significativa — conta-se apenas 1 atraso de transmissão (o suficiente para "abrir espaço" ao fluxo de pacotes).

Exemplo resolvido

Cenário (b) com números concretos — WAN é o estrangulamento. Seja $R_{LAN} = 10$ Mbps, $R_{WAN} = 1$ Mbps, $l_{LAN} = 1$ km, $l_{WAN} = 1000$ km, $L = 1000$ bits por pacote.
Atrasos elementares:

$$d_{trans,LAN} = \frac{1000}{10 \times 10^6} = 0{,}1 \text{ ms} \qquad d_{prop,LAN} = \frac{1000}{2 \times 10^8} = 0{,}005 \text{ ms}$$

$$d_{trans,WAN} = \frac{1000}{1 \times 10^6} = 1 \text{ ms} \qquad d_{prop,WAN} = \frac{1\,000\,000}{2 \times 10^8} = 5 \text{ ms}$$

Traçando a passagem de cada pacote, um a um (sem saltar nenhum). Para cada pacote:

- **fim LAN**: o instante em que acaba de sair de T1 ($k \times 0{,}1$ ms);
- **chega a R1**: fim LAN + $d_{prop,LAN}$ (0,005 ms);
- **início WAN**: o **máximo** entre "chegou a R1" e "a WAN ficou livre" (o fim WAN do pacote anterior). Se a WAN ainda está ocupada, o pacote espera na fila de R1;
- **fim WAN**: início WAN + $d_{trans,WAN}$ (1 ms);
- **chega a T2**: fim WAN + $d_{prop,WAN}$ (5 ms).

Tempos em ms:

Pacote	Fim LAN	Chega a R1	Início WAN	Fim WAN	Chega a T2
1	0,1	0,105	0,105	1,105	**6,105**
2	0,2	0,205	máx(0,205; 1,105) = 1,105	2,105	**7,105**
3	0,3	0,305	máx(0,305; 2,105) = 2,105	3,105	**8,105**
4	0,4	0,405	máx(0,405; 3,105) = 3,105	4,105	**9,105**

Repara como, a partir do pacote 2, o "início da transmissão na WAN" já não é logo que o pacote chega da LAN — tem de esperar que a WAN, que é lenta, acabe de transmitir o pacote anterior (é a fila de espera a formar-se no *router* R1, por causa do estrangulamento na WAN). O último pacote (4) chega a T2 aos **9,105 ms**.
Confirmação pela fórmula:

$$d_{total} = 0{,}1 + 0{,}005 + 4 \times 1 + 5 = \mathbf{9{,}105} \text{ ms}$$

Bate certo com a última linha da tabela.

Exemplo resolvido

Cenário (a), para comparação — LAN é o estrangulamento. Trocando as capacidades: $R_{LAN} = 1$ Mbps, $R_{WAN} = 10$ Mbps (mesmas distâncias e tamanho de pacote de antes).

$$d_{trans,LAN} = 1 \text{ ms} \qquad d_{prop,LAN} = 0{,}005 \text{ ms}$$
$$d_{trans,WAN} = 0{,}1 \text{ ms} \qquad d_{prop,WAN} = 5 \text{ ms}$$

Agora é a **LAN** que acumula os pacotes (a WAN, sendo rápida, escoa-os assim que chegam). Pela fórmula do cenário (a):

$$d_{total} = 4 \times 1 + 0{,}005 + 0{,}1 + 5 = \mathbf{9{,}105} \text{ ms}$$

(Dá o mesmo valor do cenário anterior por coincidência dos números escolhidos — o que importa reter não é o valor final, mas **qual ligação recebe o fator 4**: é sempre a mais lenta, porque é aí que os pacotes se acumulam.)

Pratica agora — Store-and-forward (Ex. 3 e 4)

- **Ex. 4** — estruturalmente idêntico ao cenário LAN/WAN acima (uma LAN seguida de uma WAN, com diagrama temporal): calcula d_{trans} e d_{prop} de cada ligação, depois 1 pacote, 10 pacotes, e repete com as capacidades trocadas na (d). Repara qual das ligações passa a acumular pacotes.
- **Ex. 3** — o vídeo de 700 MB por dois routers: primeiro inteiro (store-and-forward de um bloco só), depois em pedaços de 1000 B (o *pipelining* do exemplo acima), e por fim com 20 B de cabeçalho por pedaço. Cuidado com MB vs. Mb.

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Intensidade de tráfego e explosão do atraso

Definição

Seja R a capacidade da ligação (b/s), L o comprimento médio dos pacotes (bits) e a a taxa média de chegada de pacotes (chegadas independentes umas das outras). Define-se a **intensidade de tráfego** (ou tráfego oferecido) como La/R — a fração da capacidade da ligação que, em média, está a ser pedida.

- $La/R \approx 0$: a fila está quase sempre vazia, tempo médio de espera mínimo.
- $La/R \to 1$: o tempo médio de espera na fila **aumenta muito rapidamente** (a curva não é linear — cresce de forma explosiva perto de 1).
- $La/R > 1$: chega, em média, mais "trabalho" do que aquele que a ligação consegue escoar — o tempo médio de espera tende para **infinito** (a fila cresce sem limite, na teoria; na prática a fila tem capacidade finita, e os pacotes começam a perder-se — ver secção seguinte).

Atenção

Nota adicional (não estava explícito nos slides): este comportamento não-linear é uma propriedade geral de sistemas de filas de espera com chegadas aleatórias (é estudado em teoria das filas, por exemplo em modelos M/M/1) — a ideia chave para reter é que **não há uma relação proporcional simples** entre "quão perto de 100% de utilização" está uma ligação e "quanto atraso" isso causa. Utilizar uma ligação a 95% de capacidade pode já implicar atrasos de fila muito maiores do que utilizá-la a 80%, e não apenas ligeiramente maiores. É por isso que, na prática, se tenta manter as ligações de rede com alguma margem de folga, em vez de as operar sempre no limite da capacidade.

Pratica agora — Filas de espera (Ficha 2, Ex. 3)

F2.3 A máquina de café demora 15 s a tirar um café.

- **(a)** Chegam 10 pessoas ao mesmo tempo, com a máquina livre: quanto esperam, em média, até chegar a sua vez? E no mínimo, e no máximo?
- **(b)** Com chegadas independentes, há algum tempo médio entre chegadas que garanta que ninguém espera?
- **(c)** Com chegadas independentes a uma pessoa a cada 15 s em média ($La/R = 1$), o tempo de espera médio cresce sem limite. Porquê, se em média a máquina dá conta do recado?

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Atrasos reais: traceroute

O programa `traceroute` mede o atraso real desde a origem até cada *router* ao longo do percurso para um destino. Para cada *router* i no caminho, envia 3 pacotes que chegarão a esse *router*; o *router* i devolve uma mensagem de resposta à origem; o emissor mede o tempo entre o envio e a chegada da resposta (para cada um dos 3 pacotes, dando 3 medições por salto).

Exemplo resolvido

Como ler uma saída de `traceroute`. No exemplo dos slides (`traceroute` do DCC/FCUP até `www.cmu.edu`), cada linha tem: número do salto, as 3 medições de atraso (em ms), e o nome/IP do *router* nesse salto. Coisas a reparar:

- Os primeiros saltos (dentro da rede da universidade e da rede académica portuguesa/europeia) têm atrasos muito baixos (1-6 ms) — são saltos fisicamente próximos.
- Entre os saltos 10 e 11, o atraso salta de ~32ms para ~102ms — um aumento súbito de ~70ms. Isto é o sinal característico de uma **ligação trans-oceânica** (fibra submarina Europa-EUA): a distância é muito maior, logo o atraso de **propagação** (l/v) aumenta muito nesse único salto (não é o processamento nem a fila que sobem — é a distância física).
- A linha com `* * * Request timed out` significa que não houve resposta a nenhum dos 3 pacotes enviados àquele *router* naquele salto — pode ser porque o *router* está configurado para não responder a este tipo de pedido, ou porque o pacote (ou a resposta) se perdeu. Não significa necessariamente que a rede esteja quebrada ali — o `traceroute` normalmente continua a conseguir sondar os saltos seguintes.

Pratica agora — ping e traceroute (Ficha 2, Ex. 7)

F2.7

- **(a)** Para que serve e como funciona o `ping`? Que informação dá?
- **(b)** Para que serve o `traceroute` e como funciona? (Pensa em como é que o *router* número i, e só ele, é "obrigado" a responder.)
- **(c)** Corre `traceroute -U -n -N 1 www.cmu.edu`, localiza cada salto com uma ferramenta de geolocalização de IPs na web, e relaciona a distância com os RTTs medidos.

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Perdas de pacotes

Como a fila de espera de um *router* tem **capacidade finita**, um pacote que chegue e encontre a fila cheia é **descartado** — é uma perda de pacote. Um pacote perdido pode:

- ser retransmitido pelo nó anterior (se esse nó implementar essa garantia — nem todos os protocolos de ligação o fazem);
- ser retransmitido pelo terminal de origem (é o que o TCP faz, ao nível de transporte, detetando a perda e reenviando);
- ou simplesmente não ser retransmitido (aceitável para tráfego que usa UDP, como vídeo em tempo real, em que chegar tarde não compensa).

Pratica agora — Perdas e melhor esforço (Ficha 2, Ex. 4 e 6)

F2.4 O problema dos **dois generais**: os mensageiros podem ser apanhados (como os pacotes num serviço de melhor esforço, que se podem perder). Prova que não existe nenhum protocolo que garanta que os dois generais ficam **ambos** certos da hora de ataque, ou encontra um contra-exemplo. Pista: supõe que existe um protocolo correto com o **menor** número possível de mensagens, e pensa na **última** mensagem.

F2.6 Na demonstração de perdas em filas de espera (taxa de transmissão fixa, chegadas com distribuição exponencial):

- **(a)** Com a taxa de emissão máxima e a de transmissão mínima, ao fim de quanto tempo acontece a primeira perda?
- **(b)** Repetindo a experiência, o valor é o mesmo? Porquê?

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Throughput (débito)

Definição — Débito (throughput)

O **débito** é a taxa a que os bits são efetivamente transferidos entre o emissor e o recetor. Distingue-se o débito **instantâneo** (num instante específico) do débito **médio** (ao longo de um período de tempo mais alargado).
Se pensarmos na transferência de dados como um fluido a passar por um "tubo": se o servidor envia dados a uma taxa R_s e o cliente os recebe a uma taxa R_c, e essas taxas forem diferentes ao longo do percurso, o débito efetivamente observado é limitado pelo **elo mais lento** do percurso — a **ligação de estrangulamento** (*bottleneck link*): a ligação, ao longo de todo o caminho, que limita o débito final do fluxo (independentemente de R_s ou R_c serem maiores, se houver algures no meio uma ligação mais lenta, é ela que dita o débito).

Exemplo resolvido

Débito com várias conexões a partilhar um *backbone*. Suponha-se que existem 10 conexões, cada uma entre um servidor com débito de acesso $R_s = 2$ Mbps e um cliente com débito de acesso $R_c = 2$ Mbps, mas todas as 10 conexões partilham de forma justa uma ligação de *backbone* comum com capacidade $R = 10$ Mbps.
O débito por conexão é dado por $\min(R_c, R_s, R/10)$ — o mínimo entre a capacidade de acesso do cliente, a do servidor, e a fatia do *backbone* que cabe a cada uma das 10 conexões (assumindo partilha igual entre todas):

$$\min(2, 2, 10/10) = \min(2, 2, 1) = \mathbf{1} \text{ Mbps}$$

Ou seja, apesar de cliente e servidor terem, cada um, capacidade para 2 Mbps, o débito real fica limitado a 1 Mbps por conexão — o *backbone* partilhado é que é, aqui, a ligação de estrangulamento. Isto mostra por que motivo, na prática, R_c ou R_s (as ligações de acesso, mais próximas do utilizador) nem sempre são a ligação de estrangulamento — o gargalo pode estar bem mais "no meio" da rede, num ponto partilhado por muitos fluxos ao mesmo tempo.

Atenção

Nota adicional (não estava explícito nos slides): produto largura de banda-atraso. Define-se como $R \cdot d_{prop}$ — o produto entre a capacidade da ligação e o atraso de propagação. Fisicamente, representa o **número máximo de bits que podem estar "em trânsito" na ligação num dado instante**, antes de o primeiro bit enviado chegar ao recetor — é como perguntar "quantos bits cabem dentro do tubo, se o tubo tivesse o comprimento da ligação?".
Exemplo: uma ligação com $R = 100$ Mbps e um atraso de propagação de $d_{prop} = 15$ ms (por exemplo, uma ligação de fibra de ~3000 km, com $v = 2 \times 10^8$ m/s: $d_{prop} = 3\,000\,000/(2 \times 10^8) = 0{,}015$s). O produto largura de banda-atraso é:

$$R \cdot d_{prop} = 100 \times 10^6 \text{ b/s} \times 0{,}015 \text{ s} = 1\,500\,000 \text{ bits} = 1{,}5 \text{ Mbit}$$

Isto significa que, no instante em que o primeiro bit enviado está prestes a chegar ao recetor, o emissor já pode ter colocado até 1,5 Mbit "dentro do tubo", ainda a caminho, sem que nenhuma confirmação de receção tenha ainda voltado. Este conceito será central mais tarde, quando se estudar o dimensionamento da janela de transmissão do TCP (para o emissor conseguir "encher o tubo" e maximizar o débito, sem desperdiçar capacidade à espera de confirmações).

Pratica agora — Débito e produto largura de banda-atraso (Ex. 2 e 5)

- **Ex. 2** (o camião com discos entre o Porto e Amesterdão) — débito = bits transportados / tempo; na (b) compara o **atraso**, não a capacidade.
- **Ex. 5** — d_{trans}, d_{prop} e $R \cdot d_{prop}$ para 1 Mbps e depois 1 Gbps. A (e), "comprimento de um bit", é v/R. A (g) é a interpretação da nota adicional acima.

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Pratica agora — Estimar a capacidade de uma ligação (Ficha 2, Ex. 5)

F2.5 Envia-se dois datagramas grandes, de tamanho igual L, um logo a seguir ao outro, e mede-se o tempo entre a primeira e a segunda resposta.

- **(a)** Faz o diagrama temporal da geração, transmissão e receção dos dois pacotes (como os diagramas de *store-and-forward* acima).
- **(b)** Deduz a fórmula da capacidade a partir do tempo medido. Pista: na ligação mais lenta, o segundo pacote tem de esperar que o primeiro acabe de ser transmitido.
- **(c)** Que fatores podem estragar a estimativa, e como os atenuar?

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Camadas protocolares e modelos de serviço

Porquê estruturar em camadas?

As redes são sistemas complexos: muitas "peças" diferentes (terminais, *routers*, ligações de tipos diferentes, aplicações, protocolos, hardware, software). A pergunta natural é: como tornar essa complexidade tratável?

Exemplo resolvido

Analogia: produção e venda de roupa. Pensa numa cadeia de produção de roupa: produção das fibras (matéria-prima) → fabricação das peças → *design*, *branding* e controlo de qualidade → venda a retalho → cliente. Cada camada depende apenas do serviço fornecido pela camada imediatamente abaixo (a loja de retalho não precisa de saber como as fibras são produzidas, só precisa que a fábrica lhe entregue peças de roupa acabadas). É uma série de passos desde a matéria-prima até ao consumidor, organizados numa **estrutura por camadas**.

Vantagens de organizar um sistema complexo em camadas:

- Uma estrutura explícita permite identificar e relacionar as diferentes peças do sistema — serve como **modelo de referência** para discussão (todos falam da "camada de transporte" e sabem do que se trata, sem ambiguidade).
- A modularização facilita a manutenção e atualização do sistema: alterações na **implementação** de uma camada são transparentes para o resto do sistema, desde que o **serviço** que ela presta às camadas vizinhas se mantenha igual (tal como uma mudança na organização interna de uma fábrica não afeta o resto da cadeia de produção de roupa, desde que continue a entregar as mesmas peças).

A pilha protocolar da Internet

Definição — As 5 camadas da pilha da Internet

De cima para baixo:

1. **Aplicação:** suporta as aplicações em rede — protocolos como FTP, SMTP, HTTP.
2. **Transporte:** transferência de dados entre *processos* (não só entre máquinas) — protocolos TCP e UDP (ver Aula 2, "Serviços de transporte da Internet: TCP e UDP").
3. **Rede:** encaminhamento dos **datagramas** desde a origem até ao destino — protocolo IP e protocolos de encaminhamento.
4. **Ligação de dados:** transferência de dados **direta** entre elementos de rede adjacentes (ex: entre um terminal e o *switch* a que está ligado, ou entre dois *routers* vizinhos) — Ethernet, WiFi, PPP.
5. **Física:** os sinais no meio físico que representam os *bits*.

O modelo de referência ISO/OSI

O modelo OSI tem **7 camadas** — as mesmas 5 da Internet, mais duas entre a aplicação e o transporte:

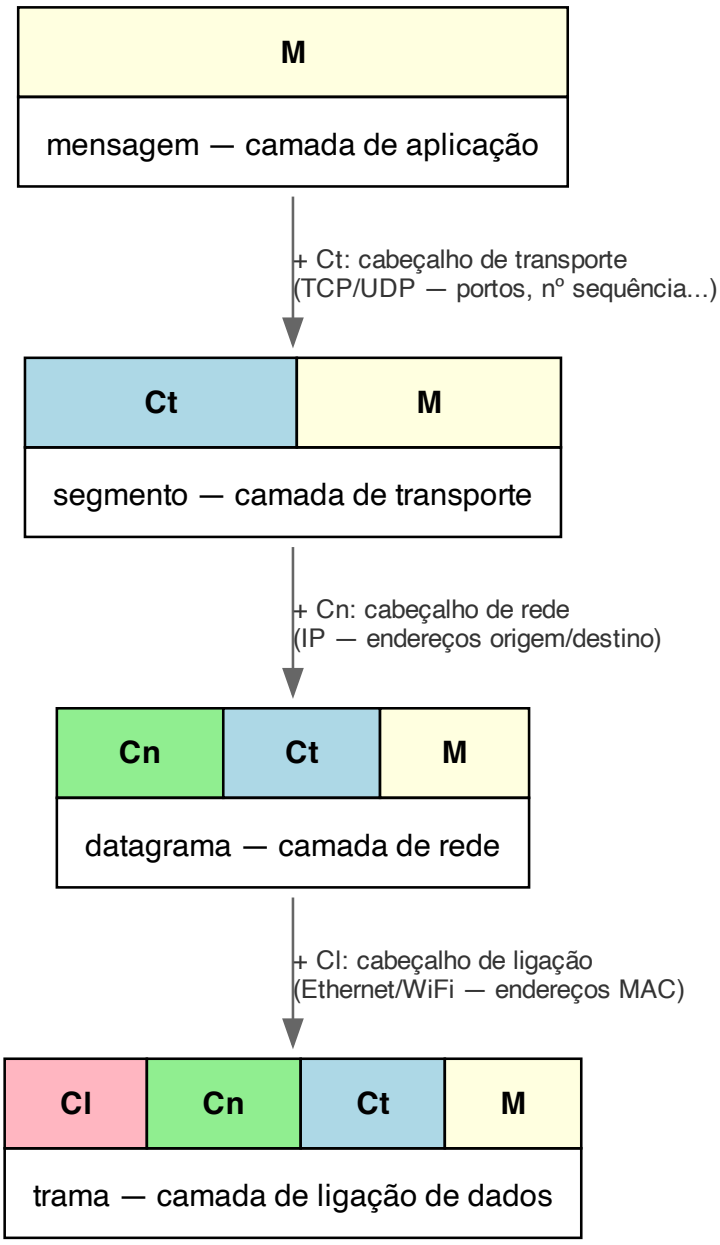
- **Apresentação:** permite às aplicações interpretar o significado dos dados — por exemplo, compressão, cifragem, convenções específicas da máquina (como representar números, texto, etc.).
- **Sessão:** autenticação, sincronização, *checkpointing* e recuperação de sessões.

Atenção

Nota adicional (não estava explícito no slide, mas foi dito explicitamente que é importante): a pilha da Internet **não tem** estas duas camadas (apresentação e sessão) como camadas próprias e separadas. Se uma aplicação precisar destes serviços (por exemplo, cifrar os dados antes de enviar, como faz o HTTPS), esses serviços têm de ser **implementados dentro da própria camada de aplicação** — não existe uma camada dedicada na pilha da Internet que os forneça automaticamente a todas as aplicações. É uma diferença de design deliberada entre os dois modelos, e é um ponto fácil de confundir num exame (perguntar "que camada do modelo OSI corresponde à camada de aplicação da Internet?" não tem uma resposta de um-para-um simples, porque a aplicação da Internet cobre também o que no OSI seria apresentação e sessão).

Encapsulamento

Quando uma mensagem desce a pilha protocolar no terminal de origem, cada camada acrescenta o seu próprio **cabeçalho** de controlo aos dados recebidos da camada acima — é o processo de **encapsulamento**. No terminal de destino, o processo é inverso: cada camada remove o seu cabeçalho antes de entregar os dados à camada de cima.


M
mensagem — camada de aplicação
+ Ct: cabeçalho de transporte
(TCP/UDP — portos, nº sequência...)
Ct
M
segmento — camada de transporte
+ Cn: cabeçalho de rede
(IP — endereços origem/destino)
Cn
Ct
M
datagrama — camada de rede
+ Cl: cabeçalho de ligação
(Ethernet/WiFi — endereços MAC)
Cl
Cn
Ct
M
trama — camada de ligação de dados

Exemplo resolvido

Seguindo uma mensagem pela rede, passo a passo. A camada de aplicação do terminal de origem entrega uma **mensagem** (M) à camada de transporte. A camada de transporte acrescenta o seu cabeçalho (C_t — por exemplo, os números de porto de origem/destino do TCP), formando um **segmento** ($C_t\,M$), e entrega-o à camada de rede. A camada de rede acrescenta o seu cabeçalho (C_n — os endereços IP de origem e destino), formando um **datagrama** ($C_n\,C_t\,M$), e entrega-o à camada de ligação de dados. A camada de ligação acrescenta o seu cabeçalho (C_l — os endereços MAC de origem e destino), formando uma **trama** ($C_l\,C_n\,C_t\,M$), pronta para ser enviada como sinais na camada física.
Ao longo do percurso, os dispositivos intermédios só "abrem" os cabeçalhos de que precisam para fazer o seu trabalho: um **switch** (dispositivo da camada de ligação de dados) só olha para o cabeçalho C_l, para decidir por que porta física reenviar a trama — não sabe nem precisa de saber o que está dentro do datagrama IP. Um **router** (dispositivo da camada de rede) tem de "abrir" a trama (removendo C_l) para chegar ao datagrama e ler o cabeçalho C_n (o endereço IP de destino), decidir para que ligação de saída o deve reenviar, e depois **voltar a encapsular** o datagrama numa nova trama (com um novo C_l, apropriado à ligação de saída seguinte) — é por isso que os endereços MAC mudam a cada salto ao longo do percurso, enquanto os endereços IP (dentro de C_n) se mantêm os mesmos do início ao fim.
Só no terminal de destino é que todos os cabeçalhos são sucessivamente removidos (ligação → rede → transporte) até restar apenas a mensagem M original, entregue à aplicação de destino.

Frequente em exame

Um erro comum é pensar que o *router* olha para o cabeçalho de **transporte** (C_t) para decidir o encaminhamento — não olha: o encaminhamento na camada de rede é feito exclusivamente com base no cabeçalho de **rede** (C_n, endereço IP de destino). Da mesma forma, um *switch* nunca olha para o endereço IP — só usa endereços MAC (camada de ligação). Esta separação de responsabilidades por camada é um ponto frequentemente testado.

Pratica agora — Arquitetura em camadas (Ficha 2, Ex. 1)

F2.1 Um *router* congestionado tem de descartar pacotes. O Chico propõe que o *router* olhe para o **cabeçalho de transporte** de cada pacote para ver se é email, e que dê prioridade ao email. A Teresa, que é purista em arquitetura protocolar, concorda com o objetivo mas não com o método. Porquê? Se o Chico puder alterar o **cabeçalho da camada de rede**, como pode agradar à Teresa? Que problemas pode ter essa solução? (Usa o que está acima sobre que camadas implementa um *router*.)
Soluções, uma alínea de cada vez: `SOLUCOES.html`

Aula 2 — Camada de aplicação

Capítulo 2 do livro (slides em `Teoricas/Aula_03.pdf`). A Aula 1 deu a visão geral da Internet e das camadas. Este capítulo sobe ao topo da pilha: como se organizam as **aplicações em rede**, que **serviço de transporte** pedem, e como funcionam os protocolos de aplicação mais comuns: **HTTP** (Web), **FTP**, **SMTP/POP3/IMAP** (email), **DNS**, o P2P do **BitTorrent**, e como se programa uma aplicação com **sockets**.

Os objetivos dos slides: perceber os aspetos conceptuais e de implementação dos protocolos de aplicação (modelos de serviço da camada de transporte, paradigmas cliente-servidor e *peer-to-peer*), aprender os conceitos através de protocolos reais, e programar aplicações com a API de sockets.

Princípios das aplicações em rede

Onde corre uma aplicação em rede

Exemplos de aplicações em rede: e-mail, web, mensagens instantâneas, terminal remoto, partilha de ficheiros P2P, jogos multi-utilizador, *streaming* de vídeo, voz sobre IP, videoconferência em tempo real, computação *grid*, redes sociais.

Criar uma aplicação em rede é escrever programas que **correm nos terminais** e **comunicam através da rede** (por exemplo, o software do servidor Web comunica com o software do browser). Escreve-se **muito pouco software para os dispositivos do núcleo da rede**: os *routers* não correm aplicações de utilizador (só implementam as camadas de rede para baixo). Manter as aplicações nos terminais permite **desenvolvê-las e divulgá-las depressa**: para lançar uma aplicação nova não é preciso mudar nada nos *routers* da Internet.

Arquiteturas das aplicações

Há três arquiteturas: **cliente-servidor**, ***peer-to-peer* (P2P)** e **híbridas** das duas.

Definição — Arquitetura cliente-servidor

Servidor: está **sempre ligado**, tem um **endereço IP fixo**, e para escalar (servir mais clientes) replica-se em vários servidores, o que tem **custo**.
Cliente: comunica com o servidor; **não precisa de estar sempre ligado**; pode ter um **endereço IP dinâmico**; **não comunica diretamente com outros clientes**.
Exemplos: um browser a pedir uma página a um servidor Web; um cliente de email a falar com um servidor de email.

Definição — Arquitetura P2P pura

- **Não há servidores dedicados**: os terminais comunicam **diretamente** entre si, e cada um faz ora de cliente, ora de servidor.
- Os pares (*peers*) **podem não estar sempre ligados** e **podem mudar de endereço IP**.
- **Muito escalável**: cada par novo traz **procura** (quer ficheiros) mas também **oferta** (serve ficheiros aos outros), ao mesmo tempo.
- **Difícil de gerir**, precisamente porque os nós se ligam de forma intermitente e os endereços mudam.

Definição — Arquiteturas híbridas

Misturam os dois modelos. Exemplos dos slides:

- **BitTorrent** (partilha de ficheiros): um **servidor central (*tracker*)** serve para descobrir os endereços IP dos pares que estão a partilhar o ficheiro; a troca dos pedaços do ficheiro é **direta entre pares**, sem o *tracker*.
- **Mensagens instantâneas**: as mensagens de texto vão **diretamente** entre os utilizadores (P2P), mas há um **serviço central** de deteção de presença e localização: cada utilizador regista o seu IP no servidor quando se liga, e usa o servidor para saber o estado e o IP dos seus contactos.

Atenção

Nota adicional (não estava explícito nos slides): a distinção cliente/servidor vs. P2P não é sobre *quem inicia* a ligação: em ambos os casos alguém tem de a iniciar. A diferença está na **arquitetura**: no modelo cliente/servidor há uma entidade fixa e sempre disponível (o servidor) de que todos os clientes dependem; no P2P não há esse ponto central, e o sistema continua a funcionar mesmo que alguns pares saiam. É por isso que o BitTorrent distribui ficheiros grandes sem um servidor central potente (ver "Aplicações P2P", abaixo, com as contas).

Processos, endereçamento e sockets

Definição — Processos cliente e servidor

Um **processo** é uma instância de um programa a correr numa máquina. Dois processos na **mesma** máquina comunicam com os mecanismos de IPC do sistema operativo; processos em máquinas **diferentes** comunicam **trocando mensagens** através da rede.

- **Cliente**: o processo que **inicia ativamente** a comunicação.
- **Servidor**: o processo que **espera passivamente** ser contactado.

Nas aplicações P2P, cada processo é **ao mesmo tempo** cliente e servidor.

Definição — Endereçamento de processos

Para receber mensagens, um processo precisa de um **identificador**. Cada terminal tem um **endereço IP** de 32 bits, mas **o IP não chega** para identificar o processo, porque na mesma máquina correm **muitos processos**. O identificador é o par **(endereço IP, número de porta)**: a porta identifica o processo dentro da máquina.

Exemplos de portas: servidor **HTTP: 80**; servidor de **email: 25**. Para enviar um pedido HTTP ao servidor Web `www.dcc.fc.up.pt` usa-se o IP 193.136.39.12 e a porta 80.

Definição — Socket

Um processo envia e recebe mensagens através de uma **socket**: é a "porta" entre o processo e o protocolo de transporte. Escrever numa socket é dizer ao sistema operativo que envie a mensagem; o processo **assume que existe uma infraestrutura de transporte** que a leva até à socket do processo recetor.

- Acima da socket: controlado pelo **programador da aplicação**.
- Abaixo da socket (TCP/UDP com os seus *buffers* e variáveis): controlado pelo **sistema operativo**.

A API permite (1) **escolher o protocolo de transporte** e (2) **alterar alguns parâmetros** dele. A programação com sockets está no fim deste capítulo.

O que define um protocolo de aplicação

Definição — Protocolo de aplicação

Um protocolo de aplicação define:

- os **tipos de mensagens** trocadas (por exemplo, pedido e resposta);
- a **sintaxe** das mensagens: que campos contêm e como se separam;
- a **semântica** das mensagens: o significado da informação em cada campo;
- as **regras** de como e quando os processos reagem às mensagens.

Protocolos de domínio público estão definidos em **RFCs** e permitem a **interoperação** entre sistemas diferentes (ex.: HTTP, SMTP). **Protocolos proprietários** só são acessíveis a quem o detentor dos direitos decidir (ex.: Zoom).

Que serviço de transporte precisa uma aplicação

As aplicações diferem em quatro requisitos:

- **Perdas**: algumas toleram algumas perdas (ex.: áudio); outras exigem uma transferência **100% fiável** (ex.: transferência de ficheiros).
- **Latência**: algumas exigem **atrasos baixos** para funcionar bem (ex.: voz sobre IP, jogos em rede).
- **Débito**: algumas precisam de um **débito mínimo** (ex.: multimédia); outras, as **aplicações elásticas**, usam o débito que houver.
- **Segurança**: integridade dos dados, privacidade (cifragem), autenticação das partes.

Aplicação	Perdas	Débito	Sensível aos atrasos
transferência de ficheiros	não tolera	elástica	não
e-mail	não tolera	elástica	não
web	não tolera	elástica	não
áudio/vídeo em tempo real	tolera	áudio 5 kbps–1 Mbps; vídeo 10 kbps–5 Mbps	sim, centenas de ms
áudio/vídeo armazenado	tolera	idem	sim, alguns segundos
jogos interativos	tolera	desde alguns kbps	sim, centenas de ms
mensagens instantâneas	não tolera	elástica	parcialmente

Serviços de transporte da Internet: TCP e UDP

A Internet oferece às aplicações **dois** serviços de transporte, com propriedades muito diferentes (a Aula 1 apresentou-os; aqui fica a versão completa).

Definição — Serviço TCP (Transmission Control Protocol, RFC 793)

- **Orientado a conexões**: exige **estabelecimento prévio da conexão** entre emissor e recetor (*handshaking*: cria-se "estado" nos dois lados, usado durante toda a comunicação).
- **Transporte fiável** entre os processos: nenhum dado se perde (uma perda é detetada e retransmitida), e os dados chegam **pela ordem** em que foram enviados, como um **fluxo contínuo de bytes** (*byte-stream*).
- **Controlo de fluxo**: o emissor não envia mais depressa do que o recetor consegue processar.
- **Controlo de congestionamento**: o emissor **reduz o débito** quando a rede está sobrecarregada.
- **Não tem**: **delineação de mensagens** (a aplicação tem de saber onde acaba cada mensagem dentro do fluxo de bytes), **garantias de atraso máximo** nem de **débito mínimo**.

Definição — Serviço UDP (User Datagram Protocol, RFC 768)

- **Transporte não fiável** entre os processos: os dados podem perder-se (e não são retransmitidos).
- **Delineação de mensagens**: **um datagrama UDP corresponde a uma mensagem** (ao contrário do TCP).
- **Não tem**: estabelecimento prévio de conexão, fiabilidade, controlo de fluxo, controlo de congestionamento, garantias de atraso máximo ou de débito mínimo.

Nem todas as aplicações precisam das garantias do TCP.

Atenção

Nota adicional (não estava explícito nos slides): é fácil pensar "o TCP é sempre melhor porque tem mais garantias", mas essas garantias têm um custo: o *handshaking*, as retransmissões e o controlo de fluxo/congestionamento trazem **atraso extra**. Numa chamada de voz, 200 ms à espera de uma retransmissão são piores do que ignorar o pacote perdido; por isso essas aplicações usam UDP. A escolha é um compromisso entre **fiabilidade** e **atraso/simplicidade**. Repara também que **nenhum** dos dois dá garantias de atraso ou de débito: essas não existem na Internet.

Aplicação	Protocolo de aplicação	Transporte
e-mail	SMTP [RFC 2821]	TCP
terminal remoto	Telnet [RFC 854]	TCP
web	HTTP [RFC 2616]	TCP
transferência de ficheiros	FTP [RFC 959]	TCP
streaming multimédia	HTTP (ex.: YouTube), RTP [RFC 1889]	TCP ou UDP
voz sobre IP	SIP, RTP, proprietários (ex.: Zoom)	tipicamente UDP

Segurança: TLS

O **TCP e o UDP não cifram nada**: as *passwords* enviadas em texto claro (*cleartext*) atravessam a Internet **visíveis** a quem capturar os pacotes (vê-se isso na captura FTP da secção FTP, abaixo).

O **SSL/TLS** acrescenta **cifragem/privacidade**, **integridade dos dados** e **autenticação das extremidades** (os **certificados** garantem a identidade do servidor). O TLS funciona **na camada de aplicação**: é uma biblioteca que a aplicação usa e que "fala" com o TCP, com uma API de sockets própria; as *passwords* continuam a atravessar a Internet, mas **cifradas**. **O SSL é obsoleto e foi substituído pelo TLS.**

Web e HTTP

Páginas, objetos e URLs

Uma **página web** é um **ficheiro-base HTML** que inclui uma série de **objetos referenciados**. Os objetos podem ser ficheiros HTML, imagens JPEG, sons, vídeos, etc. Cada objeto é endereçável por um **URL**:

```
http://www.dcc.fc.up.pt/~rprior/photo.jpg
------ ---------------- -----------------
protocolo   servidor         caminho
```

Visão geral do HTTP

Definição — HTTP (HyperText Transfer Protocol)

Protocolo de aplicação da Web, no modelo **cliente/servidor**:

- **cliente**: o browser, que pede, recebe e mostra os objetos;
- **servidor**: o servidor Web, que envia os objetos em resposta aos pedidos (ex.: Apache).

Versões: HTTP/1.0 (RFC 1945), HTTP/1.1 (RFC 2068), HTTP/2 (RFC 9113), HTTP/3 (RFC 9114).
Usa o TCP: o cliente inicia uma conexão TCP para a **porta 80** do servidor; o servidor aceita-a; trocam-se mensagens HTTP; a conexão TCP é fechada.
O HTTP é *stateless* (sem estado): o servidor **não guarda nenhuma informação** sobre os pedidos anteriores do cliente.

Atenção

Porque é que ser *stateless* é uma vantagem? Os protocolos que mantêm estado são **complexos**: é preciso memória para o estado, e se o servidor ou o cliente forem abaixo, as suas visões do estado podem ficar **inconsistentes** e ter de ser ressincronizadas. Um servidor sem estado trata cada pedido de forma independente.

Conexões não persistentes e persistentes

Definição — Persistência das conexões

- **HTTP não persistente**: numa conexão TCP só se transmite **um único objeto**. É o HTTP/1.0.
- **HTTP persistente**: enviam-se **vários objetos na mesma conexão TCP**. Por isso é preciso definir **onde termina cada mensagem** (é o cabeçalho `Content-Length`, abaixo). O HTTP/1.1 usa normalmente conexões persistentes.

Atenção

Nota adicional (não estava explícito nos slides): quem decide se a conexão fica aberta. É o cabeçalho `Connection:` e a versão do pedido:

- **HTTP/1.0**: por omissão o servidor **fecha a conexão** depois de enviar a resposta. Só fica aberta se o cliente pedir `Connection: keep-alive` e o servidor aceitar (foi uma extensão posterior ao 1.0).
- **HTTP/1.1**: por omissão a conexão é **persistente**: o servidor responde e fica à espera do pedido seguinte (até um *timeout*). Quem quiser fechar diz `Connection: close`: o servidor envia essa última resposta e fecha.

Como o servidor não fecha, o cliente **só sabe onde acaba cada resposta pelo `Content-Length`** (ou pelo `Transfer-Encoding: chunked`). Isto vê-se bem a fazer pedidos à mão (Ficha 3, Ex. 1, abaixo).

Exemplo resolvido — HTTP não persistente, passo a passo (slides)

O utilizador escreve o URL `www.dcc.fc.up.pt/~rprior/homepage/index.html` (uma página com texto e referências para 10 imagens JPEG).

1. **(1a)** O cliente HTTP inicia uma conexão TCP para o processo servidor na porta 80 de `www.dcc.fc.up.pt`. **(1b)** O servidor, que estava à espera de conexões na porta 80, aceita-a e avisa o cliente.
2. O cliente envia pela socket uma **mensagem de pedido** com o URL: quer o objeto `/~rprior/homepage/index.html`.
3. O servidor recebe o pedido, gera uma **mensagem de resposta** com o objeto e envia-a pela sua socket.
4. O servidor **fecha a conexão TCP**.
5. O cliente recebe a resposta com o ficheiro HTML e mostra-o. Ao interpretar o HTML, encontra as referências às 10 imagens.
6. Os passos 1 a 5 **repetem-se para cada uma das 10 imagens**.

Definição — RTT e tempo de resposta

RTT (*round-trip time*): tempo de ida e volta de um **pequeno pacote** entre o cliente e o servidor. Tempo de resposta de um objeto em HTTP não persistente:

- **1 RTT** para iniciar a conexão TCP (o cliente pede, o servidor responde);
- **1 RTT** para o pedido HTTP e os primeiros bytes da resposta;
- o **tempo de transmissão** do ficheiro.

$$\text{total} = 2\,\text{RTT} + \text{tempo de transmissão do ficheiro}$$

Problemas do HTTP não persistente: exige **2 RTTs por objeto**, e há *overhead* no sistema operativo para **cada** conexão TCP. Para compensar, os browsers usam muitas vezes **várias conexões TCP paralelas** para ir buscar os objetos.

Definição — HTTP persistente, com e sem pipelining

No **HTTP persistente**, o servidor deixa a conexão **aberta** para os pedidos seguintes, que vão pela conexão já estabelecida.

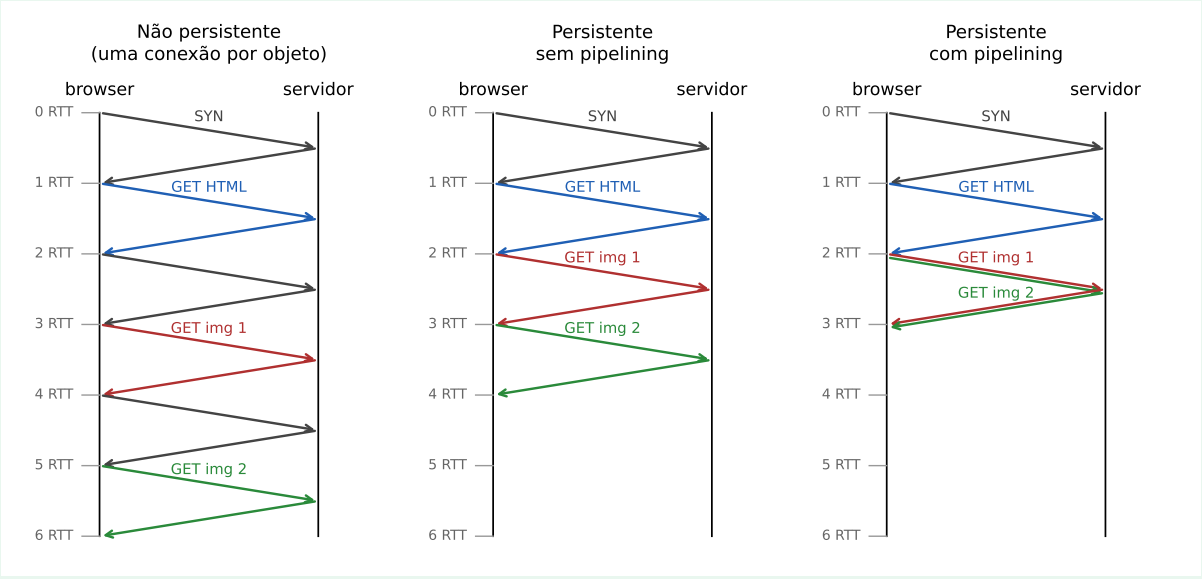
- **Sem *pipelining***: o cliente só faz um pedido novo **depois de receber a resposta anterior**: **1 RTT por cada objeto** referenciado.
- **Com *pipelining*** (o padrão no HTTP/1.1): o cliente envia o pedido **logo que encontra a referência** ao objeto, sem esperar pelas respostas anteriores: **um só RTT para todos** os objetos referenciados.

Exemplo resolvido — Quantos RTTs para uma página com 10 imagens?

Nota adicional (contas feitas para este resumo, com o exemplo dos slides). Página: 1 HTML + 10 imagens. Para ver só o efeito dos RTTs, ignora-se o tempo de transmissão (objetos pequenos).

- **Não persistente, em série**: cada um dos 11 objetos custa 2 RTT (conexão + pedido): $11 \times 2 =$ **22 RTT**.
- **Persistente sem *pipelining***: 1 RTT para a conexão + 1 RTT para o HTML (2 RTT); depois 1 RTT por imagem, uma de cada vez: $2 + 10 =$ **12 RTT**.
- **Persistente com *pipelining***: os mesmos 2 RTT até ter o HTML; depois os 10 pedidos seguem de uma vez, e as respostas chegam todas ao fim de mais 1 RTT: $2 + 1 =$ **3 RTT**.

Com tempo de transmissão T por objeto soma-se $11T$ nos três casos (os objetos passam todos pela mesma ligação, um de cada vez). A figura mostra o mesmo para 1 HTML + 2 imagens (6, 4 e 3 RTT):



Pratica agora — Tempo de resposta do HTTP (exercício inventado)

C2.1 Uma página tem um ficheiro HTML e 5 objetos referenciados. O RTT entre o browser e o servidor é 40 ms e cada objeto (incluindo o HTML) demora 10 ms a transmitir. Calcula o tempo até o browser ter tudo, com: **(a)** HTTP não persistente, um objeto de cada vez; **(b)** HTTP persistente sem *pipelining*; **(c)** HTTP persistente com *pipelining*. Usa a fórmula $2\,\mathrm{RTT} + T$ e o raciocínio do exemplo acima.
Soluções, uma alínea de cada vez: `SOLUCOES.html`

Formato das mensagens HTTP

Há dois tipos de mensagens HTTP: **pedido** e **resposta**. Ambas em **ASCII** (formato legível por humanos).

Definição — Pedido HTTP

```
GET /~rprior/index.html HTTP/1.1      <- linha de pedido
Host: www.dcc.fc.up.pt                <- linhas de cabeçalho
User-agent: Mozilla/4.0
Accept: text/html, */*;q=0.9
Accept-language: pt
                                      <- linha vazia: fim da mensagem
```

Formato genérico:

- **linha de pedido**: `método URL versão` + CR LF;
- **linhas de cabeçalho**: `nome: valor` + CR LF, uma por cabeçalho;
- **uma linha vazia** (um CR LF extra): marca o fim dos cabeçalhos;
- **corpo da mensagem** (entidade), opcional (usado, por ex., no `POST`).

Nota adicional: no **HTTP/1.1 o cabeçalho `Host:` é obrigatório** (o mesmo IP pode servir vários sites, e o servidor precisa de saber qual). Um pedido 1.1 sem `Host:` recebe `400 Bad Request` (testado em `neverssl.com`). No HTTP/1.0 é opcional.

Definição — Métodos HTTP

HTTP/1.0:

- **GET**: pede um recurso ao servidor.
- **POST**: envia ao servidor dados para processar (no corpo).
- **HEAD**: pede ao servidor **só os cabeçalhos** da resposta (sem o objeto).

HTTP/1.1: GET, POST, HEAD, e ainda

- **PUT**: faz *upload* de um ficheiro para o URL indicado;
- **DELETE**: apaga o ficheiro indicado no URL.

Definição — Resposta HTTP

```
HTTP/1.1 200 OK                       <- linha de estado
Date: Thu, 06 Aug 1998 12:00:15 GMT   <- cabeçalhos
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 ...
Content-Length: 6821                  <- para saber onde acaba a mensagem
Content-Type: text/html
                                      <- linha vazia
dados dados dados ...                 <- o objeto (ex.: o ficheiro HTML)
```

A **linha de estado** tem o **código de estado** e a descrição. Alguns códigos:

- **200 OK**: sucesso; o objeto pedido vai mais abaixo na mensagem.
- **301 Moved Permanently**: o objeto mudou para outro URL, indicado no cabeçalho `Location:`.
- **400 Bad Request**: o servidor não entendeu o pedido.
- **404 Not Found**: o recurso pedido não existe no servidor.
- **505 HTTP Version Not Supported**.

Atenção

Porque é que o `Content-Length` é necessário? Com conexões persistentes, várias respostas seguem umas atrás das outras na mesma conexão TCP, e o TCP **não delimita mensagens** (é um fluxo de bytes). O cliente lê os cabeçalhos até à linha vazia, e depois lê exatamente `Content-Length` bytes: é aí que acaba o objeto e começa a resposta seguinte.

Uma sessão HTTP real

Os slides sugerem experimentar o HTTP **à mão**, com `telnet` (ligar à porta 80 e escrever o pedido), ou com o **Wireshark** a capturar o que o browser envia e recebe. A captura do professor está em

`Teoricas/Aula_03_captura_http.cap` (abre-se no Wireshark).

Atenção — Nota adicional: o telnet como cliente TCP genérico

Os slides usam o `telnet` mas não explicam o que é (a Ficha 3 pergunta-o).

- **Para que serve**: o `telnet` é o cliente do protocolo **TELNET** (RFC 854, porta 23), de **terminal remoto**: abre uma sessão de linha de comandos numa máquina remota. Uso: `telnet <máquina> [porta]` (ex.: `telnet neverssl.com 80`). Está **obsoleto** para isso porque envia tudo, *passwords* incluídas, **em texto claro**; usa-se o **SSH**, que cifra.
- **Como funciona por dentro**: abre uma **conexão TCP** para a máquina e porta indicadas (uma *socket* cliente, como o `TCPClient` em "Programação com sockets"). Depois fica num ciclo: o que o utilizador escreve no teclado é **enviado pela socket** (no fim de cada linha manda CR LF), e o que chega pela socket é **mostrado no ecrã**. Na porta 23 também negoceia opções do terminal (comandos especiais começados pelo byte 255, `IAC`); nas outras portas é só um "tubo" de texto.
- **Porque é útil em Redes**: como HTTP, SMTP, FTP e POP3 são protocolos **de texto** sobre TCP, com o `telnet` "fazemos de browser" (ou de cliente de email): escrevemos os pedidos à mão e vemos as respostas em bruto.

No macOS não vem instalado: `brew install telnet`, ou usa o `nc` (*netcat*), que faz o mesmo (`nc neverssl.com 80`; com `nc -c` envia CR LF no fim de cada linha). Para servidores que exigem **TLS** usa-se o `gnutls-cli` ou o `openssl s_client` (ver "Correio eletrónico").

Exemplo resolvido — Primeiro pedido e resposta da captura

O browser (Firefox, 192.168.50.41) abre uma conexão TCP para a porta 80 do servidor `www.dcc.fc.up.pt` (10.0.0.251) e envia:

```
GET /~rprior/ HTTP/1.1
Host: www.dcc.fc.up.pt
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:32.0) ... Firefox/32.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
```

O servidor responde (só os cabeçalhos; o HTML vem a seguir):

```
HTTP/1.1 200 OK
Date: Mon, 13 Oct 2014 17:32:24 GMT
Server: Apache/2.2.14 (Mandriva Linux/PREFORK-1.4mdv2010.0)
Last-Modified: Mon, 17 Sep 2012 19:11:08 GMT
Content-Length: 2144
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html
```

- `Host:` diz qual o site pedido (o mesmo servidor pode servir vários).
- `Connection: keep-alive` é o browser a pedir uma **conexão persistente**; o servidor aceita (`Connection: Keep-Alive`).
- `Keep-Alive: timeout=5, max=100`: o servidor fecha a conexão ao fim de 5 s sem pedidos, ou depois de mais 100 pedidos (nota adicional: este cabeçalho não está nos slides).
- `Content-Length: 2144`: o HTML tem 2144 bytes; é assim que o browser sabe onde ele acaba.

Atenção — Nota adicional: ler conexões TCP no Wireshark

Ainda não demos o TCP (Cap. 3), mas para as capturas basta isto:

- Cada conexão TCP é identificada pelo **par de portas** (a do cliente muda de conexão para conexão; a do servidor é a 80, 21, 25…).
- Uma conexão **abre** com um pacote **SYN** do cliente (o servidor responde com SYN-ACK) e **fecha** com pacotes **FIN** (um de cada lado). Filtro: `tcp.flags.syn==1 && tcp.flags.ack==0` mostra só as aberturas.
- *Statistics* → *Conversations* → *TCP* lista as conexões, com início e duração. Botão direito num pacote → *Follow* → *TCP Stream* mostra o diálogo de texto dessa conexão.
- Conexões **simultâneas** são as que estão abertas ao mesmo tempo (o SYN de uma chega antes do FIN da outra). Uma conexão é **persistente** se transporta **mais do que um pedido** antes do FIN.

Pratica agora — HTTP à mão (Ficha 3, Ex. 1)

F3.1 (`Praticas/Semana_3/ficha_3_protocolos_aplicacionais.pdf`). Usa a nota sobre o `telnet` e a de "quem decide se a conexão fica aberta".

- **(a)** Para que serve o `telnet` e o que tem o utilizador de fazer?
- **(b)** Como funciona por dentro (3 ou 4 frases)?
- **(c)** Liga-te à porta 80 de `neverssl.com` e envia os 5 pedidos do enunciado (o 1.º em HTTP/1.0; os outros em HTTP/1.1, o último com `Connection: close`). Repara **quando é que a conexão fecha**.
- **(d)** Explica conexões persistentes e *pipelining* e comenta o que viste. Pista: experimenta também colar **vários pedidos de uma vez**, antes de chegar a primeira resposta.

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Pratica agora — Capturas HTTP (Ficha 3, Ex. 7a e 7d)

F3.7(a) Abre `Teoricas/Aula_03_captura_http.cap` (é a `Cap2-http.cap` do Moodle) no Wireshark, com o filtro `http`.

- **(i)** Identifica o método e o URL de **todos** os pedidos.
- **(ii)** Com os SYN e os FIN, descobre quantas conexões **simultâneas** foram usadas e se são **persistentes**.

F3.7(d) Faz a tua captura (filtro de captura `port 80`) enquanto visitas `http://neverssl.com/`. **(i)** O que observas? **(ii)** Foram usadas conexões persistentes?

C2.2 (inventado, complementa o 7a, na mesma captura): **(a)** que códigos de estado aparecem, e para que objetos? **(b)** Estima o RTT a partir do SYN do cliente e da resposta do servidor. O que diz esse valor sobre onde está o cliente?

Soluções, uma alínea de cada vez: `SOLUCOES.html`

HTTP/2 e HTTP/3

Definição — Limitações do HTTP/1.1: bloqueio HOL

Mesmo com conexões persistentes e *pipelining*, o HTTP/1.1 tem limitações de desempenho:

- os objetos são entregues **pela ordem em que foram pedidos**, por isso um objeto **grande** atrasa todos os que foram pedidos depois: **bloqueio *head-of-line* (HOL)**;
- a **recuperação de perdas** (retransmissões do TCP) atrasa **todos** os objetos.

Exemplo dos slides: o cliente pede O1 (grande) e depois O2, O3, O4 (pequenos). No HTTP/1.1, O2, O3 e O4 **têm de esperar** que O1 acabe.

Definição — HTTP/2

Mantém os **métodos, códigos de resposta e semântica** do HTTP/1.1, mas:

- usa **codificação binária**, com **multiplexagem de objetos**: cada objeto é dividido em **tramas**, e as tramas dos vários objetos são transmitidas **intercaladas**. No exemplo, O2, O3 e O4 chegam depressa, sem esperar por O1: **elimina o bloqueio HOL**;
- suporta **priorização de objetos** (ex.: as imagens na parte visível da página são mais urgentes do que as que só aparecem mais abaixo);
- permite ***server push***: o servidor envia objetos **que não foram pedidos** (ex.: a folha de estilo CSS, que vai ser pedida de certeza).

Definição — HTTP/3 e QUIC

O HTTP/2 ainda tem limitações:

- com **uma só conexão TCP**, a recuperação de uma perda **atrasa todos os objetos** (o TCP entrega os bytes por ordem). Isto incentiva a usar várias conexões paralelas, o que dificulta o controlo de congestionamento;
- a **segurança obriga a um RTT adicional**: primeiro o *handshake* do TCP (transporte), depois o do TLS (segurança), em sequência.

O **HTTP/3** resolve isto com outro protocolo de transporte, o **QUIC** (*Quick UDP Internet Connections*): está **implementado na camada de aplicação e funciona sobre UDP**, com **mecanismos próprios** de recuperação de erros e de controlo de congestionamento.

	HTTP/2 sobre TCP	HTTP/3
Aplicação	HTTP/2 + TLS	HTTP/2 (simplificado) + QUIC
Transporte	TCP	UDP
Rede	IP	IP

- **Estabelecimento da conexão**: com TCP+TLS há **2 *handshakes* sequenciais** (TCP para a fiabilidade e o controlo de congestionamento, TLS para a autenticação e a cifragem). O QUIC faz **1 *handshake* para tudo**.
- **0-RTT**: no fim da primeira conexão, o servidor envia um ***ticket* de sessão TLS**, que o cliente guarda em *cache*. Na conexão seguinte, o cliente usa esse *ticket* para obter logo a informação de autenticação e cifragem, e **envia dados logo no primeiro pacote**: o RTT inicial desaparece.

Web caches (servidores *proxy*)

Definição — Web cache (proxy)

Objetivo: **satisfazer os pedidos dos clientes sem recorrer ao servidor original**.

- O utilizador configura o browser para usar o *proxy*; o browser envia **todos** os pedidos HTTP ao *proxy*.
- Se o objeto está em *cache*, o *proxy* devolve-o **diretamente**.
- Senão, o *proxy* pede-o ao servidor original, devolve-o ao browser e **guarda uma cópia** em *cache*.

O *proxy* é **cliente e servidor** ao mesmo tempo (servidor para o browser, cliente para o servidor original). Normalmente é instalado por uma organização (universidade, empresa, ISP), e pode também **registar ou filtrar** os pedidos. O próprio browser também faz *caching*.

Porquê fazer *caching*? Reduz o **tempo de resposta** aos pedidos; reduz o **tráfego no acesso à Internet** da organização; e, havendo muitas *caches*, até servidores de baixa capacidade funcionam bem, porque os **acessos repetidos à mesma página** são servidos pelas *caches*.

Pratica agora — À volta do HTTP: o proxy do Paulo (Ficha 3, Ex. 2)

F3.2 O browser do Paulo só fala **HTTP/1.0** (ligação a 10 Mbps). Ele instala um *proxy* **HTTP/1.1** sem *cache* nem compressão. Browser e *proxy* só têm uma sessão HTTP de cada vez; há um só servidor.

- **(a)** Quantifica o ganho no tempo total para n objetos de tamanho m, com o *proxy* no próprio computador e sem atrasos de processamento.
- **(b)** E se o *proxy* estivesse noutro computador, com um RTT browser–*proxy* grande?
- **(c)** E se o browser suportasse HTTP/1.1, com o *proxy* no computador dele?

Pista: o *proxy* parte o caminho em **dois troços**. Escreve o tempo de cada objeto com $2\,\mathrm{RTT} + m/R$ (não persistente) ou $\mathrm{RTT} + m/R$ (persistente), como no exemplo dos 22/12/3 RTT, e pensa: o *proxy* pode fazer *pipelining* se o browser só lhe pede um objeto de cada vez?
Soluções, uma alínea de cada vez: `SOLUCOES.html`

FTP

Definição — FTP (File Transfer Protocol, RFC 959)

Transfere ficheiros **de/para** um servidor remoto, no modelo cliente/servidor: o **cliente** é o lado que desencadeia a transferência (em qualquer sentido); o **servidor** é a máquina remota. Porta do servidor: **21**.

Definição — Conexões de controlo e de dados

- O cliente liga-se ao servidor na **porta 21** (TCP): é a **conexão de controlo**. Nela faz a **autenticação** e **navega** no sistema de ficheiros remoto, enviando comandos.
- Quando o servidor recebe um comando de **transferência de ficheiro**, **abre outra conexão TCP** (a partir da sua **porta 20**) para transferir o ficheiro: a **conexão de dados**. No fim da transferência, é fechada.
- Para transferir outro ficheiro, o servidor abre **outra** conexão de dados.

O controlo é ***out of band***: vai numa conexão **separada** dos dados. E o servidor FTP **mantém estado** (o diretório corrente, a autenticação), ao contrário do HTTP.

Definição — Comandos e respostas FTP

Enviados como texto ASCII na conexão de controlo (tal como no HTTP).

Comando	Significado
`USER username`	indica o utilizador
`PASS password`	indica a *password*
`PORT`	indica o IP e a porta do cliente para a conexão de dados
`LIST`	lista os ficheiros do diretório corrente
`RETR filename`	recebe um ficheiro do servidor
`STOR filename`	envia um ficheiro para o servidor

As respostas têm um **código de estado e uma descrição**, como no HTTP: `331 Username OK, password required`; `125 data connection already open; transfer starting`; `425 Can't open data connection`; `452 Error writing file`.
Nota adicional: na captura do professor aparecem mais quatro comandos que os slides não listam: `SYST` (pergunta o tipo de sistema do servidor), `CWD dir` (muda o diretório corrente), `TYPE I` (passa a modo binário, "*image*", para o ficheiro chegar byte a byte; `TYPE A` é o modo texto) e `QUIT` (termina a sessão).

Atenção

Nota adicional (não estava explícito nos slides): como se lê o `PORT`. O argumento são 6 números: os 4 primeiros formam o IP do cliente, e os 2 últimos a porta, como $p_1 \times 256 + p_2$ (a porta tem 16 bits: p_1 é o byte mais significativo). Por exemplo `PORT 10,0,0,5,4,1` quer dizer IP 10.0.0.5, porta $4 \times 256 + 1 = 1025$: é para aí que o servidor liga a conexão de dados. A resposta a um `PORT` às vezes sugere "use PASV": no modo **passivo** (`PASV`) é o **cliente** que abre a conexão de dados, o que funciona melhor quando o cliente está atrás de uma *firewall* ou NAT.

Pratica agora — Captura FTP (Ficha 3, Ex. 7b)

F3.7(b) Abre `Teoricas/Aula_03_captura_ftp.cap` (a `Cap2-ftp.cap` do Moodle) no Wireshark (filtro `ftp || ftp-data`).

- **(i)** Identifica **todos** os comandos enviados ao servidor e diz para que serve cada um.
- **(ii)** Em que pacote é pedido o estabelecimento da conexão de dados? (Descodifica o argumento com a nota sobre o `PORT`.)
- **(iii)** Que portas usa o servidor para a conexão de controlo e para a de dados? Quem abre cada uma?

C2.3 (inventado, complementa o 7b, na mesma captura): **(a)** que utilizador e *password* foram usados, e o que mostra isto sobre a segurança do FTP? **(b)** Qual o tamanho do ficheiro e o débito médio da transferência (do primeiro ao último pacote com dados da conexão de dados)?
Soluções, uma alínea de cada vez: ***`SOLUCOES.html`***

Correio eletrónico

Definição — Componentes do e-mail

Três componentes:

- **Agentes de utilizador** (AU): o leitor/cliente de email (Mozilla Thunderbird, Outlook, …), para escrever, editar e ler mensagens. As mensagens a enviar e as recebidas ficam no servidor.
- **Servidores de email**: cada um tem uma **caixa de correio** por utilizador (as mensagens recebidas) e uma **fila de mensagens de saída** (por enviar).
- O protocolo **SMTP** (*Simple Mail Transfer Protocol*), usado **entre servidores** para enviar as mensagens. Na conversa entre dois servidores, o "cliente" é o servidor que **envia** e o "servidor" é o que **recebe**.

Definição — SMTP (RFC 2821)

- Usa o **TCP** para transferir a mensagem com fiabilidade para o servidor, que está à escuta na **porta 25**.
- A transferência é **direta** do servidor do remetente para o servidor do destinatário.
- Três fases: ***handshaking***, **transferência das mensagens**, **fecho da conexão**.
- Interação **comando/resposta**: os comandos são texto ASCII; as respostas são um código de estado e a descrição.
- As mensagens têm de estar em **ASCII de 7 bits**: caracteres acentuados e anexos binários **têm de ser codificados**.

Exemplo resolvido — A Alice envia uma mensagem ao Bob (slides)

1. No seu AU, a Alice escreve a mensagem e o endereço do destinatário, `bob@someschool.edu`.
2. O AU da Alice envia a mensagem para o **servidor da Alice**, que a põe na **fila de saída**.
3. O "lado cliente" do SMTP no servidor da Alice **abre uma conexão TCP** para o **servidor do Bob**.
4. A mensagem é enviada por essa conexão.
5. O servidor do Bob põe a mensagem na **caixa de correio** do Bob.
6. O Bob usa o seu AU para ler a mensagem.

Exemplo resolvido — Diálogo SMTP dos slides

`S:` é o servidor (`hamburger.edu`), `C:` o cliente (`crepes.fr`):

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

- `220`: o servidor apresenta-se quando a conexão abre.
- `HELO`: o cliente identifica-se (*handshaking*).
- `MAIL FROM` / `RCPT TO`: remetente e destinatário (um `RCPT TO` por destinatário).
- `DATA`: a seguir vem a mensagem, que acaba numa linha só com `.`.
- `QUIT`: fecha a conexão.

Os slides propõem experimentar à mão: `telnet smtp.fc.up.pt 25`, esperar pelo `220` e usar `HELO`, `MAIL FROM`, `RCPT TO`, `DATA` e `QUIT`. Assim envia-se uma mensagem sem nenhum cliente de email.

Definição — Notas sobre o SMTP e comparação com o HTTP

- O SMTP usa **conexões persistentes** (várias mensagens na mesma conexão).
- Exige que a mensagem (cabeçalho e corpo) esteja em **ASCII de 7 bits** (daí a pergunta: como enviar anexos binários? Resposta: MIME, abaixo).
- Usa `CRLF.CRLF` (uma linha só com um ponto) para marcar o **fim da mensagem**.

	HTTP	SMTP
sentido	***pull***: o cliente vai buscar	***push***: o cliente empurra
comandos e respostas	ASCII	ASCII
códigos de resposta	sim, com descrição	sim, com descrição
objetos	um objeto por resposta	vários objetos numa mensagem *multiparte*

Definição — Formato das mensagens (RFC 2822) e MIME

O SMTP é o protocolo de **troca**; o formato das **mensagens** é o RFC 2822:

- **cabeçalhos**, por exemplo `To:`, `From:`, `Subject:`. Atenção: são **diferentes dos comandos SMTP** (`MAIL FROM`, `RCPT TO`);
- uma **linha vazia**;
- o **corpo**, a "mensagem" (só caracteres ASCII).

MIME (extensão multimédia do email, RFC 2045, 2056): linhas extra no cabeçalho declaram o tipo de conteúdo:

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0                    <- versão do MIME
Content-Transfer-Encoding: base64    <- método de codificação
Content-Type: image/jpeg             <- tipo/subtipo (e parâmetros)

base64 encoded data .....            <- dados codificados em ASCII
```

Exemplo resolvido — Uma sessão SMTP real (captura do professor, 1/2)

`Teoricas/Aula_03_captura_smtp.cap`: o Thunderbird (10.0.0.92) envia uma mensagem pelo servidor `smtp.dcc.fc.up.pt` (10.0.0.10, porta 25). Depois do *handshake* TCP:

```
S: 220 smtp.dcc.fc.up.pt ESMTP Postfix (2.1.4) (Mandrake Linux)
C: EHLO [10.0.0.92]
S: 250-smtp.dcc.fc.up.pt
S: 250-PIPELINING
S: 250-SIZE 10240000
S: 250-VRFY
S: 250-ETRN
S: 250-STARTTLS
S: 250 8BITMIME
C: MAIL FROM:<rprior@dcc.fc.up.pt> SIZE=639
S: 250 Ok
C: RCPT TO:<rcprior@fc.up.pt>
S: 250 Ok
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
```

- `EHLO` em vez de `HELO`: é o SMTP estendido (ESMTP). O servidor responde com a **lista de extensões** que suporta, uma por linha (`250-` quer dizer "há mais linhas"; `250` com espaço é a última). Nota adicional: o ESMTP não está nos slides.
- `STARTTLS`: o servidor aceita passar a conversa para TLS (cifrada).
- `SIZE=639`: o cliente avisa o tamanho da mensagem.

Exemplo resolvido — Uma sessão SMTP real (captura do professor, 2/2)

Depois do `354`, o cliente envia a mensagem (cabeçalhos RFC 2822, linha vazia, corpo), a linha com o `.`, e fecha:

```
Message-ID: <48E66E0E.50205@dcc.fc.up.pt>
Date: Fri, 03 Oct 2008 20:10:06 +0100
From: Rui Prior <rprior@dcc.fc.up.pt>
User-Agent: Thunderbird 2.0.0.17 (Windows/20080914)
MIME-Version: 1.0
To: rcprior@fc.up.pt
Subject: Mensagem de teste
Content-Type: text/plain; charset=ISO-8859-1
Content-Transfer-Encoding: 8bit

Olá!
Isto é apenas uma curta mensagem de teste ...
.
S: 250 Ok: queued as 588D7E837A
C: QUIT
S: 221 Bye
```

(Alguns cabeçalhos foram omitidos.) Repara que o endereço aparece **duas vezes**: no **comando** `RCPT TO` (o "envelope", que é o que o servidor usa para entregar) e no **cabeçalho** `To:` (parte da mensagem, que o leitor mostra). E o corpo tem acentos com `Content-Transfer-Encoding: 8bit`: só é permitido porque o servidor anunciou a extensão `8BITMIME`; sem ela, a regra dos 7 bits obrigava a codificar o texto.

Atenção — Nota adicional: SMTP à mão com TLS (STARTTLS)

Hoje quase nenhum servidor aceita SMTP em texto claro. Com **STARTTLS** a conexão começa normal na porta 25 (`220`, `EHLO`), o cliente envia `STARTTLS`, e a partir daí **tudo passa a ir cifrado com TLS**, na mesma conexão TCP. Depois disso o cliente tem de mandar **outra vez** `EHLO`: o diálogo anterior "não conta" (e a lista de extensões muda; na FCUP passa a aparecer `AUTH LOGIN PLAIN`).

Como já não dá para escrever texto em claro, o `telnet` não serve. O `gnutls-cli` faz de `telnet` com TLS:

```
gnutls-cli smtp.fc.up.pt -p 25 --starttls-proto=smtp --crlf
```

- `--starttls-proto=smtp`: o próprio `gnutls-cli` faz o `EHLO` e o `STARTTLS` em claro e o *handshake* TLS. Quando te dá o controlo, já estás dentro do TLS: começas por `EHLO`.
- `--crlf`: o Enter envia CR LF (o SMTP exige CR LF no fim das linhas).
- Alternativa: `openssl s_client -starttls smtp -crlf -connect smtp.fc.up.pt:25`. Cuidado: no `s_client` uma linha que começa por `R` maiúsculo pede renegociação TLS e uma por `Q` fecha a conexão. Escreve `rcpt to:` e `quit` em minúsculas (o SMTP aceita).

Testado em 2026-10-06: o `smtp.fc.up.pt` aceita a conexão e o STARTTLS.

Atenção — Nota adicional: o SMTP não verifica o remetente

Nada no SMTP dos slides confirma que quem envia é quem diz ser: o `MAIL FROM` e o cabeçalho `From:` são **texto escrito pelo cliente**. Por isso consegue-se enviar um email "de" `bugsbunny@warnerbros.com` (é o objetivo do Ex. 3 da Ficha 3). O TLS cifra a conversa, mas **não prova o remetente**. Hoje há mecanismos à parte que o servidor que recebe pode verificar: **SPF** (um registo DNS do domínio diz que servidores podem enviar em nome dele), **DKIM** (o servidor de origem assina a mensagem) e **DMARC** (o que fazer se falharem). Uma mensagem forjada costuma ir parar ao *spam* ou levar um aviso.

Definição — Protocolos de acesso ao email

O SMTP faz a **entrega e o armazenamento** no servidor do destinatário. Para o destinatário **ir buscar** o email à caixa de correio usa-se um **protocolo de acesso**:

- **POP** (*Post Office Protocol*, RFC 1939): autorização (agente ↔ servidor) e *download*.
- **IMAP** (*Internet Mail Access Protocol*, RFC 1730): mais funcionalidades (e mais complexo); manipula as mensagens **guardadas no servidor**.
- **HTTP**: o *webmail* (Gmail, Proton Mail, etc.).

Exemplo resolvido — Sessão POP3 (slides)

Fase de autorização: comandos `user` e `pass`; respostas `+OK` ou `-ERR`. **Fase de transações**: `list` (lista as mensagens), `retr` (transfere uma), `dele` (apaga-a do servidor), `quit` (termina).

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 2 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

(`list` dá o número e o tamanho em bytes de cada mensagem; o `.` sozinho fecha uma lista ou uma mensagem.)

Definição — POP3 vs IMAP

POP3: o exemplo acima usa o modo **"transfere e apaga"**: se o Bob mudar de cliente, **não pode reler** a mensagem. Há o modo **"transfere e deixa ficar"**, para usar vários clientes. O POP3 **não mantém estado entre sessões**.

IMAP: as mensagens **ficam no servidor**; permite organizá-las em **pastas**; **mantém estado entre sessões** (os nomes das pastas e que mensagem está em que pasta). É mais complexo.

Pratica agora — E-mail (Ficha 3, Ex. 3 e 7c)

F3.3 Com o `gnutls-cli` (nota acima), envia um email para a tua conta da FCUP com remetente `bugsbunny@warnerbros.com`, assunto "Eh… What's up, doc?" e corpo "Did you know email is insecure?". Verifica que chegou. Escreve primeiro, no papel, as linhas que vais enviar (comandos SMTP **e** cabeçalhos da mensagem) e onde fica a linha vazia.
F3.7(c) Abre `Teoricas/Aula_03_captura_smtp.cap` no Wireshark e identifica os comandos enviados e as respostas. A sessão está transcrita acima; o exercício é encontrá-la nos pacotes (filtro `smtp`, ou *Follow TCP Stream*) e dizer **em que pacote** vai cada comando.
C2.4 (inventado) O Bob lê o email no portátil e no telemóvel. Com POP3 no modo "transfere e apaga", o que vê no telemóvel depois de o portátil ir buscar o correio? Que protocolo resolve isto, e porquê?
Soluções, uma alínea de cada vez: `SOLUCOES.html`

DNS

Definição — DNS (Domain Name System)

As máquinas na Internet têm dois identificadores: o **endereço IP** (32 bits, usado no encaminhamento dos datagramas) e o **nome** (usado por humanos, ex.: `www.dcc.fc.up.pt`). O DNS faz o **mapeamento entre nomes e endereços IP**. É:

- uma **base de dados distribuída**, implementada como uma **hierarquia de servidores de nomes**;
- um **protocolo de aplicação**: as aplicações, para comunicar, **resolvem** os nomes (tradução nome → endereço). É um serviço fundamental da Internet, mas implementado nas **extremidades** da rede (a complexidade fica nas extremidades).

Serviços do DNS:

- tradução de **nomes para endereços IP**;
- **múltiplos nomes** para a mesma máquina (*aliasing*): um **nome canónico** e **pseudónimos**;
- identificação do(s) **servidor(es) de email** de um domínio;
- **distribuição de carga**: servidores replicados, com um **conjunto de endereços IP para um só nome**. (Nota adicional: o servidor devolve a lista toda, mas **muda a ordem** de resposta para resposta, em *round robin*. Como a maioria dos clientes usa o primeiro endereço da lista, clientes diferentes acabam em réplicas diferentes.)

Porque não um servidor central? Seria um **ponto único de falha**, teria um **volume de tráfego** enorme, ficaria **longe** da maioria dos clientes, e a **manutenção** seria complexa. Não seria **escalável**.

Hierarquia de servidores

Definição — Servidores de raiz, TLD e com autoridade

- **Servidores de raiz**: contactados por um servidor local que não consegue resolver um nome; indicam os servidores responsáveis pelo **domínio de topo** do nome pedido (ex.: `com`, `pt`). Para usar o DNS é preciso conhecer o IP de pelo menos um. Há **13 servidores de raiz** (letras a a m), espalhados pelo mundo e muitos deles replicados em dezenas de locais. Nota adicional: um servidor local conhece-os por um ficheiro instalado com ele, as ***root hints*** (`named.root`, publicado pela IANA/InterNIC em `https://www.internic.net/domain/named.root`), com os nomes e IPs dos
  13. Ao arrancar, pergunta a um deles "quais são os servidores da raiz?" (registos **NS** do nome `.`) e fica com a lista **atual**: basta que um dos IPs do ficheiro ainda esteja certo.
- **Servidores de domínio de topo (TLD)**: responsáveis por `com`, `org`, `net`, `edu`, etc., e pelos domínios de topo de país (`pt`, `uk`, `fr`, …). A Network Solutions mantém os de `com`; a DNS.PT os de `pt`.
- **Servidores com autoridade**: os servidores DNS **de uma organização**, que têm autoridade sobre os mapeamentos dos nomes dessa organização (web, email) para endereços IP. Podem ser mantidos pela própria organização ou por um ISP em nome dela.

Ideia-base, para obter o IP de `www.amazon.com`: perguntar a um servidor de **raiz**, que indica os servidores de `com`; perguntar a um servidor de **`com`**, que indica os servidores de `amazon.com`; perguntar a um servidor de **`amazon.com`**, que responde com o IP de `www.amazon.com`.

Definição — Resolução iterativa e recursiva

- **Iterativa**: o servidor contactado **devolve uma referência** a outro(s) servidor(es): "não sei resolver este nome, mas pergunta ao servidor X". Quem pergunta faz as perguntas todas.
- **Recursiva**: o servidor contactado devolve **sempre a resposta final** (ou um erro): passa o **ónus da resolução** para o servidor. Em toda a hierarquia **não se usa na prática**: sobrecarregaria muito os servidores, sobretudo os de raiz.

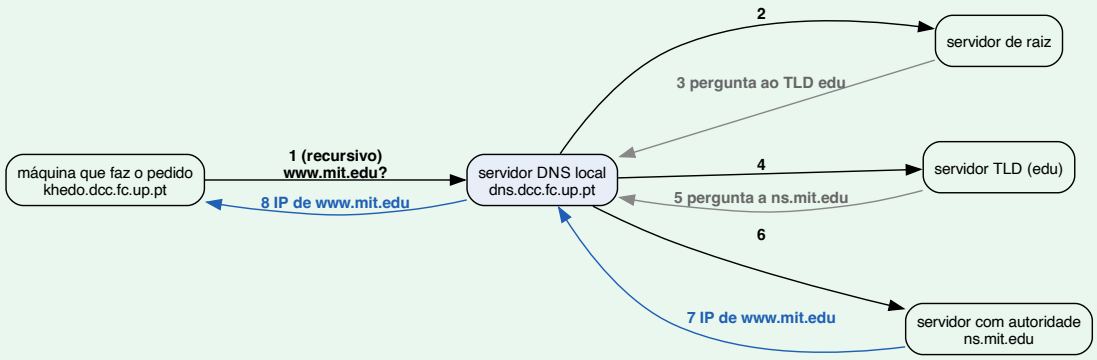
Definição — Servidor de nomes local

Não pertence estritamente à hierarquia. Cada organização (ISP, empresa, universidade) tem um ou mais. Quando uma máquina local precisa de resolver um nome, **envia o pedido ao servidor DNS local**, que atua como *proxy*:

- aceita **pedidos recursivos** das máquinas locais;
- **itera** ao longo da hierarquia até obter a tradução;
- devolve a resposta à máquina que perguntou;
- faz ***caching*** das respostas.

Normalmente um servidor DNS só aceita pedidos recursivos de máquinas locais.

Exemplo resolvido — O caso normal: khedo.dcc.fc.up.pt quer o IP de www.mit.edu



1. `khedo` faz um pedido **recursivo** ao servidor local `dns.dcc.fc.up.pt`.
2. O servidor local pergunta a um servidor de **raiz**…
3. …que responde com os servidores do TLD `edu`.
4. O servidor local pergunta ao servidor **TLD** `edu`…
5. …que responde com o servidor com autoridade de `mit.edu`, `ns.mit.edu`.
6. O servidor local pergunta a `ns.mit.edu`…
7. …que responde com o **IP de** `www.mit.edu`.
8. O servidor local devolve esse IP a `khedo` (e guarda-o em *cache*).

O pedido 1 é **recursivo**; os pedidos 2, 4 e 6 são **iterativos**. São 8 mensagens com a *cache* vazia.

Definição — Caches e atualização de registos

- Quando um servidor aprende a resolução de um nome, **guarda-a em *cache***. As entradas são removidas ao fim de um certo tempo (*timeout*, o TTL do registo).
- Nota adicional: o **TTL** é escolhido pelo **administrador do domínio** e o servidor com autoridade devolve-o sempre igual (ex.: 300 s). Um servidor local que responde **da *cache*** devolve o **tempo que falta** para a entrada expirar: o TTL vai **descendo** de pergunta para pergunta, e quando chega a 0 a entrada é apagada e a próxima pergunta volta a ir ao servidor com autoridade (o TTL "recomeça" no valor original).
- Os servidores TLD estão **quase sempre em *cache*** nos servidores locais, por isso **os servidores de raiz raramente são contactados**.
- As atualizações aos registos são normalmente feitas pelo **administrador** da rede; há mecanismos de **atualização dinâmica** (RFC 2136).

Registos e mensagens DNS

Definição — Registos de recurso (RR)

O DNS é uma base de dados distribuída de **registos de recurso**, com o formato **(nome, tipo, valor, ttl)**. O significado de nome e valor depende do tipo:

Tipo	nome	valor
A	nome da máquina	endereço IP
NS	domínio (ex.: `foo.com`)	nome de um servidor com autoridade sobre esse domínio
CNAME	um pseudónimo	o nome canónico (o nome real)
MX	domínio	nome do servidor de email desse domínio

Exemplo de CNAME: `www.up.pt` é, na realidade, `www.up.pt.cdn.cloudflare.net` (o valor do registo CNAME).

Definição — Mensagens DNS

Mensagens de **pergunta** e **resposta**, **ambas com o mesmo formato**, **sobre UDP**, em **formato binário**.

- **Cabeçalho (12 bytes)**:
  - ***identification***: número de 16 bits que permite **associar a resposta à pergunta** (necessário porque o UDP não tem conexões);
  - ***flags***: pergunta ou resposta; recursão desejada; recursão disponível; resposta com autoridade;
  - quatro contadores: número de perguntas, de RRs de resposta, de RRs de autoridade e de RRs adicionais.
- **questions**: os campos de nome e tipo da pergunta;
- **answers**: os RRs de resposta à pergunta;
- **authority**: os registos NS dos servidores com autoridade;
- **additional information**: informação adicional potencialmente útil (por exemplo, os registos A desses servidores).

Exemplo resolvido — Ler uma resposta DNS com o dig (slides)

```
$ dig mit.edu
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 28372
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 3, ADDITIONAL: 3
;; QUESTION SECTION:
;mit.edu.                  IN   A
;; ANSWER SECTION:
mit.edu.           60      IN   A    18.7.22.69
;; AUTHORITY SECTION:
mit.edu.           11809   IN   NS   STRAWB.mit.edu.
mit.edu.           11809   IN   NS   BITSY.mit.edu.
mit.edu.           11809   IN   NS   W20NS.mit.edu.
;; ADDITIONAL SECTION:
BITSY.mit.edu.     363     IN   A    18.72.0.3
W20NS.mit.edu.     3667    IN   A    18.70.0.160
STRAWB.mit.edu.    3667    IN   A    18.71.0.151
```

- `id: 28372` é o *identification*; `flags: qr rd ra`: é uma **resposta** (*qr*), a **recursão foi pedida** (*rd*) e **está disponível** (*ra*).
- Os contadores (1 pergunta, 1 resposta, 3 de autoridade, 3 adicionais) são os 4 contadores do cabeçalho.
- Cada linha é um RR: nome, **ttl** (em segundos), classe `IN` (Internet), tipo, valor. A resposta é um registo **A**: `mit.edu` tem o IP 18.7.22.69, válido em *cache* durante 60 s.
- Na secção de autoridade vêm os **NS** de `mit.edu`; na adicional, os **A** desses servidores, para não ser preciso perguntar outra vez.

Exemplo resolvido — Inserir registos no DNS (slides)

Uma empresa nova, a Network Utopia, quer o domínio `networkutopia.com`:

1. Regista o nome num **agente de registo** (*DNS registrar*, ex.: Network Solutions), dando os nomes e IPs dos seus servidores com autoridade.
2. O agente insere **dois RRs no servidor TLD** `com`: `(networkutopia.com, dns1.networkutopia.com, NS)` e `(dns1.networkutopia.com, 212.212.212.1, A)`.
3. No seu **servidor com autoridade**, a empresa cria os registos dos nomes dentro do domínio: um **A** para `www.networkutopia.com`, um **MX** para `networkutopia.com`, etc.

Pratica agora — DNS (exercício inventado)

C2.5

- **(a)** No caso normal acima (`khedo` quer `www.mit.edu`), quantas mensagens DNS são trocadas se: **(i)** as *caches* estão vazias; **(ii)** o servidor local já tem em *cache* o servidor TLD de `edu`; **(iii)** o servidor local já tem em *cache* o IP de `www.mit.edu`?
- **(b)** A empresa Ferramentas Lda registou `ferramentas.pt`. O servidor com autoridade é `dns1.ferramentas.pt` (192.0.2.53). O site é servido pela máquina `servidor1.ferramentas.pt` (192.0.2.10), mas as pessoas escrevem `www.ferramentas.pt`. O email do domínio é recebido por `mail.ferramentas.pt` (192.0.2.25). Escreve os RRs que ficam no **servidor TLD `pt`** e os que ficam no **servidor com autoridade**.

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Ferramentas: `host` e `dig`

Nota adicional: os slides mostram uma saída do `dig` mas não explicam as ferramentas. A Ficha 3 (Ex. 4–6) usa-as a fundo.

Definição — host e dig

Os dois enviam perguntas DNS e mostram a resposta. O `host` dá uma resposta curta e legível; o `dig` mostra a **mensagem DNS inteira** (cabeçalho, *flags*, as quatro secções), como no exemplo do `mit.edu`.

Comando	O que faz
`host nome`	IP (e pseudónimos, servidores de email) do nome
`host -t mx dominio`	só os registos de um tipo (aqui MX)
`dig nome`	pergunta registos A ao servidor local
`dig nome TIPO`	outro tipo: `MX`, `NS`, `CNAME`, `AAAA`…
`dig @servidor nome`	pergunta a **outro servidor** (nome ou IP)
`dig +short nome`	só os valores, sem o resto da mensagem
`dig +noall +answer nome`	só a secção de resposta
`dig +norecurse ...`	pedido **iterativo**: desliga a *flag* `rd`
`dig +trace nome`	faz a resolução iterativa desde a raiz, sozinho
`dig +qr nome`	mostra também a **pergunta** enviada

- **Flags** no cabeçalho: `qr` (é uma resposta), `rd` (recursão pedida), `ra` (recursão disponível), `aa` (*authoritative answer*: a resposta vem de um servidor **com autoridade** sobre o nome, não de uma *cache*).
- `AAAA` é o registo de endereço **IPv6** (o `A` é IPv4).
- Um registo **MX** tem um **número de preferência** antes do nome (`fc.up.pt. MX 10 mx01.up.pt.`): tenta-se primeiro o de número mais baixo; com números iguais, reparte-se a carga entre eles.
- A linha `;; SERVER:` no fim diz que servidor respondeu (o servidor local, se não usaste `@`).

Exemplo resolvido — Resolução iterativa à mão (Ficha 3, Ex. 4c), 1/2

Objetivo: resolver `www.dcc.fc.up.pt` sem o servidor local, começando no servidor de raiz `a.root-servers.net` (198.41.0.4). Com `+norecurse` cada servidor responde **só com o que sabe**. Saídas reais (6 out. 2026), cortadas.

Passo 1 — raiz. `dig @198.41.0.4 www.dcc.fc.up.pt +norecurse`

```
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 9, ADDITIONAL: 19
;; AUTHORITY SECTION:
pt.         172800  IN  NS  a.dns.pt.
pt.         172800  IN  NS  c.dns.pt.      (... mais 7 NS de pt.)
;; ADDITIONAL SECTION:
a.dns.pt.  172800  IN  A   185.39.208.1   (... os IPs dos outros)
```

`ANSWER: 0`: a raiz **não sabe** o IP. Responde com uma **referência**: os NS do TLD `pt` (autoridade) e os seus IPs (adicional). Não há `aa`. Escolhemos um, `a.dns.pt` = 185.39.208.1.

Passo 2 — TLD `pt`. `dig @185.39.208.1 www.dcc.fc.up.pt +norecurse`

```
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 5, ADDITIONAL: 11
;; AUTHORITY SECTION:
up.pt.     7200    IN  NS  ns1.up.pt.     (... ns2, ns3, ns4, ns02.fccn.pt)
;; ADDITIONAL SECTION:
ns1.up.pt. 7200    IN  A   193.137.55.30  (... os IPs dos outros)
```

Outra referência: o servidor de `pt` não conhece `fc.up.pt` nem `dcc.fc.up.pt`, mas sabe quem tem autoridade sobre **`up.pt`**. Passa logo para `up.pt`; não há um passo "`fc.up.pt`" porque a U.Porto não delegou esse subdomínio noutros servidores.

Exemplo resolvido — Resolução iterativa à mão (Ficha 3, Ex. 4c), 2/2

Passo 3 — servidor da U.Porto. `dig @193.137.55.30 www.dcc.fc.up.pt +norecurse`

```
;; flags: qr aa; QUERY: 1, ANSWER: 2, AUTHORITY: 5, ADDITIONAL: 1
;; ANSWER SECTION:
www.dcc.fc.up.pt.   300   IN  CNAME  echo.dcc.fc.up.pt.
echo.dcc.fc.up.pt.  600   IN  A      193.136.39.92
;; AUTHORITY SECTION:
dcc.fc.up.pt.     86400   IN  NS     ns1.up.pt.   (... e os outros 4)
```

Agora há **resposta** e a *flag* `aa`: este servidor tem autoridade sobre `dcc.fc.up.pt`. A resposta tem dois registos:

- **CNAME**: `www.dcc.fc.up.pt` é um **pseudónimo**; o nome canónico é `echo.dcc.fc.up.pt`;
- **A**: `echo.dcc.fc.up.pt` tem o IP **193.136.39.92**.

Como o servidor também tem autoridade sobre o nome canónico, mandou logo o A, e não foi preciso perguntar outra vez.

Resumo: 3 perguntas iterativas (raiz → `pt` → `up.pt`), as duas primeiras com referências e a última com a resposta. É o que o servidor local faz por nós quando lhe mandamos um pedido recursivo (o "caso normal" acima). Os TTL também contam a história: 172 800 s (2 dias) para os NS de `pt`, que quase nunca mudam; 300 s para o `www`.

Pratica agora — dig da name? (Ficha 3, Ex. 4)

F3.4 Usa a tabela de comandos acima. A alínea (c) está resolvida acima.

- **(a)** Com o `host`, obtém o IP de `www.dcc.fc.up.pt`.
- **(b)** Com o `dig`, obtém o IP de `www.google.com`. O que tem cada secção da resposta? Que tipos de registo aparecem e para que servem?
- **(d)** Faz a mesma resolução da (c) com `+trace`. Bate certo com os 3 passos?
- **(e)** Obtém a lista de IPs de `www.jn.pt`.
- **(f)** Repete e compara a ordem. Para que serve irem rodando?
- **(g)** Descobre o servidor de email de `fc.up.pt` e o seu IP (dois comandos).
- **(h)** Com `+qr`, compara o `id` da pergunta e da resposta, para vários nomes. Para que serve, e porque é que o DNS precisa dele? (Pensa no transporte que o DNS usa.)

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Pratica agora — TTL e servidores de raiz (Ficha 3, Ex. 5 e 6)

F3.5 Obtém os IPs de `www.google.com` (servidor local) e olha para o TTL. Repete passados uns segundos.

- **(a)** O que observas? Porquê? (Nota sobre o TTL em "Caches e atualização de registos".)
- **(b)** Repete, perguntando diretamente a um servidor de nomes da Google (descobre-o com `dig google.com NS`). Explica a diferença.

F3.6 Os servidores locais precisam da lista atual dos servidores de raiz.

- **(a)** Como se pode obter essa lista?
- **(b)** Obtém-na com o `dig`, usando essa técnica.

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Aplicações P2P

(As propriedades da arquitetura P2P pura estão em "Arquiteturas das aplicações", no início desta aula.)

Tempo de distribuição: cliente-servidor vs. P2P

Pergunta: quanto tempo demora a distribuir **um ficheiro de tamanho** F de um computador (o servidor) para N outros?

Notação: u_s é a capacidade de ***upload*** do servidor; u_i a capacidade de *upload* do cliente/par i; d_i a capacidade de ***download*** do cliente/par i. Supõe-se que o **núcleo da rede tem capacidade abundante**: os únicos limites são os acessos.

Definição — Cliente-servidor

- O servidor tem de enviar N **cópias** do ficheiro, NF bits ao débito u_s: demora **pelo menos** NF/u_s.
- O cliente i demora **pelo menos** F/d_i a fazer o *download*; o mais lento é o de menor d_i.

$$d_{CS} = \max\left\{\frac{NF}{u_s},\ \frac{F}{\min_i d_i}\right\}$$

Para N grande, o primeiro termo domina: **cresce linearmente com** N.

Definição — P2P

- O servidor tem de enviar **pelo menos uma cópia** do ficheiro: demora pelo menos F/u_s.
- O cliente i demora pelo menos F/d_i a fazer o *download*.
- No total têm de ser recebidos NF **bits** (cada par recebe o ficheiro inteiro), e a **maior taxa de envio agregada** possível é a do servidor **mais a de todos os pares**, $u_s + \sum_{i=1}^{N} u_i$ (os pares também enviam uns aos outros).

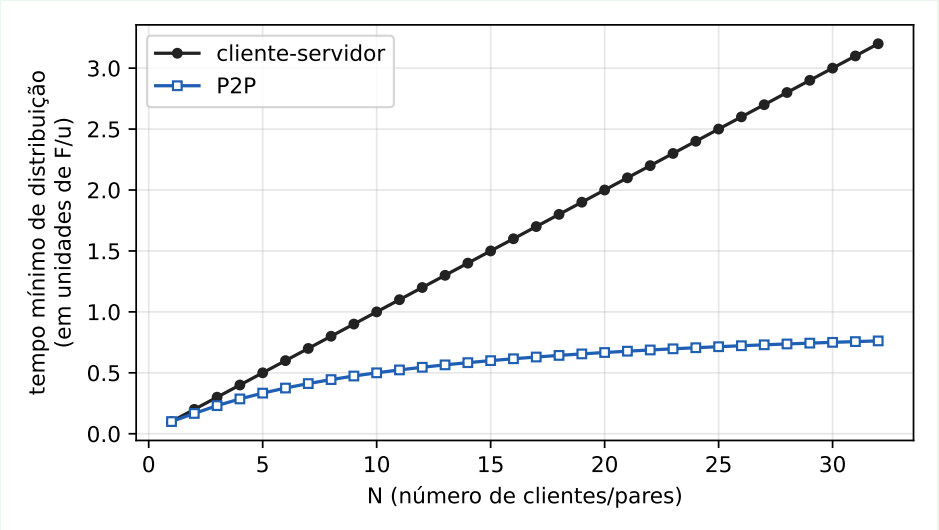
$$d_{P2P} = \max\left\{\frac{F}{u_s},\ \frac{F}{\min_i d_i},\ \frac{NF}{u_s + \sum_{i=1}^{N} u_i}\right\}$$

No último termo, **o numerador e o denominador crescem ambos com** N: cada par novo pede mais F bits, mas também traz mais u_i de capacidade.

Exemplo resolvido — O gráfico dos slides, com as contas

Nota adicional: os slides mostram o gráfico sem os valores; são os do livro. Todos os pares têm o mesmo *upload* u; $F/u = 1$ unidade de tempo; $u_s = 10u$; e o *download* é rápido ($d_{\min} \geq u_s$, logo $F/d_{\min} \leq F/u_s$ e esse termo nunca é o máximo).

- **Cliente-servidor**: $NF/u_s = N \cdot \frac{F}{10u} = \frac{N}{10}$. Para $N = 10$ dá 1; para $N = 30$ dá 3. Uma reta.
- **P2P**: $\frac{NF}{u_s + Nu} = \frac{N}{10+N}$ (dividindo tudo por u e usando $F/u = 1$), e o termo $F/u_s = 0{,}1$. Para $N = 10$: $\max\{0{,}1,\ 10/20\} = 0{,}5$. Para $N = 30$: $\max\{0{,}1,\ 30/40\} = 0{,}75$. Nunca passa de 1, por muito que N cresça ($N/(10+N) < 1$).



Pratica agora — Tempo de distribuição (exercício inventado)

C2.6 Um ficheiro de $F = 1$ Gbit vai ser distribuído a N pares. O servidor tem $u_s = 100$ Mbps de *upload*; cada par tem $u = 5$ Mbps de *upload* e $d = 50$ Mbps de *download*. Calcula d_{CS} e d_{P2P} para **(a)** $N = 10$, **(b)** $N = 100$ e **(c)** $N = 1000$. Em cada caso, diz qual é o termo que manda (o que está a limitar).
Soluções, uma alínea de cada vez: `SOLUCOES.html`

BitTorrent

Definição — BitTorrent

- Os ficheiros são divididos em **pedaços** (*chunks*, normalmente de 256 kB). Os pares enviam e recebem pedaços.
- ***Tracker***: mantém a informação sobre os pares participantes.
- ***Torrent***: o grupo de pares a trocar pedaços de um ficheiro.

Um par que **se junta** ao *torrent*:

- ainda não tem pedaços, mas vai obtê-los;
- regista-se no *tracker* para obter a lista de pares, e liga-se a um subconjunto deles (os seus **"vizinhos"**);
- **enquanto faz o *download*, vai fazendo *upload*** dos pedaços que já recebeu para outros pares;
- os pares com quem troca pedaços podem variar; entram pares novos e saem antigos (**rotatividade**);
- quando tem o ficheiro completo, pode **sair** (egoísta) ou **ficar** a enviar aos outros (altruísta).

Definição — Obtenção e envio de pedaços

Obtenção: *rarest first*. Num dado instante, pares diferentes têm partes diferentes do ficheiro. Periodicamente, cada par pergunta a cada vizinho que pedaços tem, e pede-lhe os que ainda não tem, **começando pelos mais raros** (assim os pedaços raros espalham-se depressa e não desaparecem se o único par que os tem sair).
Envio: *tit-for-tat* ("olho por olho"):

- um par envia pedaços aos **quatro vizinhos que lhe estão a enviar mais** (o "top 4"), reavaliados **a cada 10 s**;
- **de 30 em 30 s** escolhe **outro par ao acaso** e envia-lhe pedaços; esse par novo pode passar a fazer parte do "top 4" (é assim que se descobrem parceiros melhores e que os pares novos, sem nada para dar, conseguem começar).

Resultado: **quem envia mais, recebe mais**.

Programação com sockets

Definição — API de sockets

Uma **socket** é uma interface **local**, **criada pela aplicação** e **controlada pelo sistema operativo** (uma "porta"), através da qual os processos enviam e recebem mensagens de/para outros processos. A API foi introduzida em **1981**, no **BSD 4.1**. As sockets são **criadas, usadas e libertadas explicitamente**, no paradigma **cliente/servidor**.
Dois tipos de serviço de transporte:

- **não fiável, de datagramas** (UDP);
- **fiável, de sequência (*stream*) de bytes** (TCP).

Sockets TCP

Definição — Cliente e servidor TCP

Do ponto de vista da aplicação, o TCP fornece uma **transferência fiável e ordenada de bytes** entre cliente e servidor.
O servidor tem de estar à espera de ser contactado: o processo servidor tem de estar a correr, **à escuta numa socket TCP** (a *welcome socket*).
O cliente contacta o servidor: cria uma socket TCP indicando o **endereço IP e a porta** do processo servidor; ao criá-la, **estabelece uma conexão TCP** com o servidor.
Quando é contactado, o servidor cria uma socket nova (a *connection socket*) para falar com **esse** cliente. Assim o servidor pode falar com **vários clientes em simultâneo**: o IP e a porta de origem de cada um permitem distingui-los.

Servidor (em `hostid`)	Cliente
cria a socket de receção na porta `x`: `welcomeSocket = ServerSocket()`	
espera por um pedido: `connectionSocket = welcomeSocket.accept()`	cria a socket e liga a `hostid`, porta `x`: `clientSocket = Socket()` (aqui dá-se o estabelecimento da conexão TCP)
	envia o pedido pela `clientSocket`
lê o pedido da `connectionSocket`	
escreve a resposta na `connectionSocket`	
	lê a resposta da `clientSocket`
fecha a `connectionSocket`	fecha a `clientSocket`

Definição — Streams

Uma ***stream*** é uma sequência de caracteres que entra ou sai de um processo. Uma ***stream* de entrada** está associada a uma fonte de entrada (teclado ou socket); uma ***stream* de saída**, a um destino (ecrã ou socket). No exemplo seguinte, o cliente tem três: `inFromUser` (teclado), `outToServer` (para a socket) e `inFromServer` (da socket).

Aplicação de exemplo: (1) o cliente lê uma linha do teclado (`inFromUser`) e envia-a ao servidor pela socket (`outToServer`); (2) o servidor lê a linha da socket; (3) converte-a para **maiúsculas** e devolve-a; (4) o cliente lê a linha da socket (`inFromServer`) e imprime-a. Uma mensagem é **uma linha de texto**, e o terminador de mensagem é o **carácter de mudança de linha** (repara: é a aplicação que delimita as mensagens, porque o TCP não o faz).

Exemplo resolvido — Cliente TCP em Java (slides)

```java
import java.io.*;
import java.net.*;

class TCPClient {
    public static void main(String argv[]) throws Exception {
        String sentence;
        String modifiedSentence;
        // stream de entrada: o teclado
        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));
        // cria a socket e liga ao servidor (com DNS lookup implicito)
        Socket clientSocket = new Socket("hostname", 6789);
        // stream de saida associada a socket
        DataOutputStream outToServer =
            new DataOutputStream(clientSocket.getOutputStream());
        // stream de entrada associada a socket
        BufferedReader inFromServer = new BufferedReader(
            new InputStreamReader(clientSocket.getInputStream()));
        sentence = inFromUser.readLine();            // bloqueia ate ler uma linha
        outToServer.writeBytes(sentence + '\n');     // envia; '\n' termina a mensagem
        modifiedSentence = inFromServer.readLine();  // le a resposta
        System.out.println("FROM SERVER: " + modifiedSentence);
        clientSocket.close();
    }
}
```

Exemplo resolvido — Servidor TCP em Java (slides)

```java
import java.io.*;
import java.net.*;

class TCPServer {
    public static void main(String argv[]) throws Exception {
        String clientSentence;
        String capitalizedSentence;
        // socket de rececao, a escuta na porta 6789
        ServerSocket welcomeSocket = new ServerSocket(6789);
        while (true) {  // ciclo infinito: espera um cliente, atende-o, repete
            // espera que um cliente se ligue; cria uma socket nova para ele
            Socket connectionSocket = welcomeSocket.accept();
            BufferedReader inFromClient = new BufferedReader(
                new InputStreamReader(connectionSocket.getInputStream()));
            DataOutputStream outToClient =
                new DataOutputStream(connectionSocket.getOutputStream());
            clientSentence = inFromClient.readLine();           // le o pedido
            capitalizedSentence = clientSentence.toUpperCase() + '\n';
            outToClient.writeBytes(capitalizedSentence);        // escreve a resposta
            connectionSocket.close();
        }   // volta ao inicio do ciclo para esperar uma nova conexao
    }
}
```

Testado com `javac` e `java` (servidor e cliente na mesma máquina, com `"localhost"` no lugar de `"hostname"`): o cliente escreve `ola mundo` e recebe `FROM SERVER: OLA MUNDO`.

Definição — Tipos de servidores TCP

- **Iterativo** (o exemplo acima): os pedidos são processados **em sequência**; só depois de acabar um se começa o seguinte. Só serve para serviços com pedidos **esporádicos e de resposta imediata** (ex.: *daytime*).
- **Concorrente com processos**: a cada pedido de conexão, o servidor lança um **processo novo** (`fork()`) para o atender, e o processo-pai continua à escuta. Bom quando os pedidos demoram a atender mas não são demasiado frequentes (o `fork()` tem custo).
- **Concorrente com *threads***: igual, mas lança uma ***thread*** em vez de um processo. **Mais leve** em CPU, mas **menos robusto** (um erro numa *thread* pode deitar abaixo o servidor todo).
- **Multiplex**: **um só** processo/*thread* usa `select()` para atender pedidos de **várias sockets em simultâneo**. Permite relacionar clientes diferentes (ex.: um *chat*) e escutar em várias portas; usa menos memória do que os outros concorrentes; mas é **mais complexo** e **não tira partido de CPUs *multi-core***.
- ***Pre-forking***: logo no arranque, lança um certo número de processos que atendem pedidos em paralelo (com ou sem um processo coordenador que distribui a carga). Bom com **cargas elevadas**, sobretudo muitos pedidos curtos (ex.: servidor HTTP).
- ***Pre-threaded***: como o *pre-forking*, mas com *threads*: ligeiramente mais eficiente, mas menos robusto.

Sockets UDP

Definição — Cliente e servidor UDP

Não há estabelecimento de conexão (não há *handshaking*):

- o emissor indica **o IP e a porta de destino em cada pacote** enviado;
- o servidor **extrai o IP e a porta de origem** do pacote do pedido, para saber para onde enviar a resposta.

Para a aplicação, o UDP dá uma transferência **não fiável** de blocos de informação (**datagramas**): podem perder-se, chegar fora de ordem ou até **duplicados**. A diferença para o TCP: a entrada e a saída são **pacotes** (`DatagramPacket`), não uma sequência de bytes.

Servidor (em `hostid`)	Cliente
cria a socket na porta `x`: `serverSocket = DatagramSocket()`	cria a socket: `clientSocket = DatagramSocket()`
	cria o endereço (`hostid`, porta `x`) e envia o datagrama do pedido pela `clientSocket`
lê o pedido da `serverSocket`	
escreve a resposta na `serverSocket`, **indicando o IP e a porta do cliente**	
	lê a resposta da `clientSocket` e fecha-a

Exemplo resolvido — Cliente UDP em Java (slides)

```java
import java.io.*;
import java.net.*;

class UDPClient {
    public static void main(String args[]) throws Exception {
        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));
        DatagramSocket clientSocket = new DatagramSocket();
        InetAddress IPAddress = InetAddress.getByName("hostname"); // DNS
        byte[] sendData;
        byte[] receiveData = new byte[1024];
        String sentence = inFromUser.readLine();
        sendData = sentence.getBytes();
        // datagrama com os dados, o comprimento, o IP e a porta de destino
        DatagramPacket sendPacket =
            new DatagramPacket(sendData, sendData.length, IPAddress, 9876);
        clientSocket.send(sendPacket);
        DatagramPacket receivePacket =
            new DatagramPacket(receiveData, receiveData.length);
        clientSocket.receive(receivePacket);   // bloqueia ate chegar a resposta
        String modifiedSentence = new String(receivePacket.getData());
        System.out.println("FROM SERVER:" + modifiedSentence);
        clientSocket.close();
    }
}
```

Exemplo resolvido — Servidor UDP em Java (slides)

```java
import java.io.*;
import java.net.*;

class UDPServer {
    public static void main(String args[]) throws Exception {
        DatagramSocket serverSocket = new DatagramSocket(9876); // porta 9876
        byte[] receiveData = new byte[1024];
        byte[] sendData;
        while (true) {
            DatagramPacket receivePacket =
                new DatagramPacket(receiveData, receiveData.length);
            serverSocket.receive(receivePacket);          // recebe o datagrama
            String sentence = new String(receivePacket.getData());
            InetAddress IPAddress = receivePacket.getAddress(); // IP do cliente
            int port = receivePacket.getPort();                 // porta do cliente
            String capitalizedSentence = sentence.toUpperCase();
            sendData = capitalizedSentence.getBytes();
            DatagramPacket sendPacket =
                new DatagramPacket(sendData, sendData.length, IPAddress, port);
            serverSocket.send(sendPacket);
        }   // volta ao inicio para esperar um novo datagrama
    }
}
```

Atenção — Nota adicional: um erro no código UDP dos slides

Não estava nos slides; foi apanhado a **correr** o código. O servidor usa sempre o mesmo *buffer* `receiveData`, e `new String(receivePacket.getData())` converte o *buffer* **inteiro** (1024 bytes), e não só os bytes que chegaram. Resultado real: enviando primeiro `segunda mensagem mais longa` e depois `ab`, a segunda resposta é `ABGUNDA MENSAGEM MAIS LONGA` seguida de bytes nulos: o `ab` escreveu por cima das duas primeiras posições e o resto ficou da mensagem anterior. O cliente tem o mesmo problema (imprime os bytes nulos até aos 1024).
Correção, nos dois lados: usar só os bytes recebidos, `new String(receivePacket.getData(), 0, receivePacket.getLength())`. Testado: `ab` passa a dar `AB`. É a delineação de mensagens do UDP a funcionar: o datagrama sabe o seu comprimento, a aplicação é que o tem de usar.

Pratica agora — Sockets (exercício inventado)

C2.7

- **(a)** Com o `TCPServer` **iterativo** acima a correr, o cliente A liga-se mas demora 3 segundos a escrever a sua linha. Entretanto, o cliente B liga-se e envia logo a sua. Quando é que o B recebe a resposta, e porquê?
- **(b)** Transforma o `TCPServer` num servidor **concorrente com *threads***, para que o B seja atendido logo.

Soluções, uma alínea de cada vez: `SOLUCOES.html`

Resumo comparativo dos protocolos

Frequente em exame

O sumário dos slides destaca, para cada protocolo: troca de mensagens **pedido/resposta**; **formato** das mensagens (cabeçalhos + dados); mensagens de **controlo e de dados** (*in-band* ou *out-of-band*); **centralizado ou distribuído**; **com ou sem estado**; transferência **fiável ou não fiável**; e a **complexidade nas extremidades** da rede.

	HTTP	FTP	SMTP	POP3 / IMAP	DNS
transporte	TCP	TCP	TCP	TCP	UDP
porta do servidor	80	21 (dados: 20)	25	—	—
estado no servidor	não (*stateless*)	sim	—	POP3: não entre sessões; IMAP: sim	—
controlo	*in-band*	*out-of-band*	*in-band*	*in-band*	—
sentido	*pull*	ambos	*push*	*pull*	pergunta e resposta
formato	ASCII (HTTP/2: binário)	ASCII	ASCII 7 bits	ASCII	binário
arquitetura	cliente-servidor	cliente-servidor	entre servidores	cliente-servidor	distribuída, hierárquica

(—: não está nos slides; as portas do POP3, IMAP e DNS são 110, 143 e 53, mas não foram dadas.)